

Pipeline3

April 5, 2025

1 Región Pampaneira

Debido al largo contenido del primer Pipeline, que incluye un análisis exploratorio de los datos y teniendo en cuenta que hay que restringirse a la región de Pampaneira para realizar el el objetivo principal de generar datos mediante modelos transformer, se ha creado un Notebook Pipeline2.

```
[1]: import pandas as pd

pd.options.mode.chained_assignment = None

pd.reset_option('display.max_rows') # To show all rows
pd.reset_option('display.max_columns') # To show all columns
```

```
[2]: trafico_feb22_ago23 = pd.read_csv('../data/trafico_feb22_ago23.csv')
```

```
[3]: # PAM_1 y BUB
columns = ['vehicles_PAM_1_OUT',
           'vehicles_PAM_1_OUT_Zona_Granada',
           'vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras',
           'vehicles_PAM_1_OUT_Zona_Andalucia_no_GR',
           'vehicles_PAM_1_OUT_Extranjero',
           'vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid',
           'vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras',
           'vehicles_PAM_1_OUT_Zona_Otras',
           'vehicles_PAM_1_OUT_5_seats',
           'vehicles_PAM_1_OUT_+5_seats',
           'vehicles_PAM_1_OUT_-5_seats',
           'vehicles_PAM_1_OUT_nan_seats',
           'vehicles_PAM_1_OUT_101-200_CO2',
           'vehicles_PAM_1_OUT_0-100_CO2',
           'vehicles_PAM_1_OUT_nan_CO2',
           'vehicles_PAM_1_OUT_201-300_CO2',
           'vehicles_PAM_1_OUT_+300_CO2',
           'vehicles_PAM_1_IN',
           'vehicles_PAM_1_IN_Zona_Granada',
           'vehicles_PAM_1_IN_Zona_Catalunia_y_Otras',
           'vehicles_PAM_1_IN_Zona_Andalucia_no_GR',
           'vehicles_PAM_1_IN_Extranjero',
```

```
'vehicles_PAM_1_IN_Zona_Comunidad_de_Madrid',
'vehicles_PAM_1_IN_Zona_Extremadura_y_Otras',
'vehicles_PAM_1_IN_Zona_Otras',
'vehicles_PAM_1_IN_5_seats',
'vehicles_PAM_1_IN_+5_seats',
'vehicles_PAM_1_IN_-5_seats',
'vehicles_PAM_1_IN_nan_seats',
'vehicles_PAM_1_IN_101-200_CO2',
'vehicles_PAM_1_IN_0-100_CO2',
'vehicles_PAM_1_IN_nan_CO2',
'vehicles_PAM_1_IN_201-300_CO2',
'vehicles_PAM_1_IN_+300_CO2',
'vehicles_PAM_2_OUT',
'vehicles_PAM_2_OUT_Zona_Granada',
'vehicles_PAM_2_OUT_Zona_Catalunia_y_Otras',
'vehicles_PAM_2_OUT_Zona_Andalucia_no_GR',
'vehicles_PAM_2_OUT_Extranjero',
'vehicles_PAM_2_OUT_Zona_Comunidad_de_Madrid',
'vehicles_PAM_2_OUT_Zona_Extremadura_y_Otras',
'vehicles_PAM_2_OUT_Zona_Otras',
'vehicles_PAM_2_OUT_5_seats',
'vehicles_PAM_2_OUT_+5_seats',
'vehicles_PAM_2_OUT_-5_seats',
'vehicles_PAM_2_OUT_nan_seats',
'vehicles_PAM_2_OUT_101-200_CO2',
'vehicles_PAM_2_OUT_0-100_CO2',
'vehicles_PAM_2_OUT_nan_CO2',
'vehicles_PAM_2_OUT_201-300_CO2',
'vehicles_PAM_2_OUT_+300_CO2',
'vehicles_PAM_2_IN',
'vehicles_PAM_2_IN_Zona_Granada',
'vehicles_PAM_2_IN_Zona_Catalunia_y_Otras',
'vehicles_PAM_2_IN_Zona_Andalucia_no_GR',
'vehicles_PAM_2_IN_Extranjero',
'vehicles_PAM_2_IN_Zona_Comunidad_de_Madrid',
'vehicles_PAM_2_IN_Zona_Extremadura_y_Otras',
'vehicles_PAM_2_IN_Zona_Otras',
'vehicles_PAM_2_IN_5_seats',
'vehicles_PAM_2_IN_+5_seats',
'vehicles_PAM_2_IN_-5_seats',
'vehicles_PAM_2_IN_nan_seats',
'vehicles_PAM_2_IN_101-200_CO2',
'vehicles_PAM_2_IN_0-100_CO2',
'vehicles_PAM_2_IN_nan_CO2',
'vehicles_PAM_2_IN_201-300_CO2',
'vehicles_PAM_2_IN_+300_CO2',
'vehicles_BUB_OUT',
```

```

'vehicles_BUB_OUT_Zona_Granada',
'vehicles_BUB_OUT_Zona_Catalunia_y_Otras',
'vehicles_BUB_OUT_Zona_Andalucia_no_GR',
'vehicles_BUB_OUT_Extranjero',
'vehicles_BUB_OUT_Zona_Comunidad_de_Madrid',
'vehicles_BUB_OUT_Zona_Extremadura_y_Otras',
'vehicles_BUB_OUT_Zona_Otras',
'vehicles_BUB_OUT_5_seats',
'vehicles_BUB_OUT_+5_seats',
'vehicles_BUB_OUT_-5_seats',
'vehicles_BUB_OUT_nan_seats',
'vehicles_BUB_OUT_101-200_CO2',
'vehicles_BUB_OUT_0-100_CO2',
'vehicles_BUB_OUT_nan_CO2',
'vehicles_BUB_OUT_201-300_CO2',
'vehicles_BUB_OUT_+300_CO2',
'vehicles_BUB_IN',
'vehicles_BUB_IN_Zona_Granada',
'vehicles_BUB_IN_Zona_Catalunia_y_Otras',
'vehicles_BUB_IN_Zona_Andalucia_no_GR',
'vehicles_BUB_IN_Extranjero',
'vehicles_BUB_IN_Zona_Comunidad_de_Madrid',
'vehicles_BUB_IN_Zona_Extremadura_y_Otras',
'vehicles_BUB_IN_Zona_Otras',
'vehicles_BUB_IN_5_seats',
'vehicles_BUB_IN_+5_seats',
'vehicles_BUB_IN_-5_seats',
'vehicles_BUB_IN_nan_seats',
'vehicles_BUB_IN_101-200_CO2',
'vehicles_BUB_IN_0-100_CO2',
'vehicles_BUB_IN_nan_CO2',
'vehicles_BUB_IN_201-300_CO2',
'vehicles_BUB_IN_+300_CO2']

trafico_feb22_ago23["date"] = pd.to_datetime(trafico_feb22_ago23["date"])
trafico_feb22_ago23["date"] = trafico_feb22_ago23["date"].dt.tz_convert('UTC')

df_orig = trafico_feb22_ago23[["date"] + columns]
columnas_int64 = df_orig.select_dtypes(include='int64').columns

# Convertir esas columnas a float64
df_orig[columnas_int64] = df_orig[columnas_int64].astype('float64')

df_orig

```

```

[3]:
0      2022-02-04 12:00:00+00:00      0.0

```

1	2022-02-04	13:00:00+00:00	0.0
2	2022-02-04	14:00:00+00:00	0.0
3	2022-02-04	15:00:00+00:00	0.0
4	2022-02-04	16:00:00+00:00	0.0
...		...	
10025	2023-08-31	19:00:00+00:00	18.0
10026	2023-08-31	20:00:00+00:00	1.0
10027	2023-08-31	21:00:00+00:00	0.0
10028	2023-08-31	22:00:00+00:00	0.0
10029	2023-08-31	23:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Granada \		
0		0.0
1		0.0
2		0.0
3		0.0
4		0.0
...	...	
10025		8.0
10026		1.0
10027		0.0
10028		0.0
10029		0.0

vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \		
0		0.0
1		0.0
2		0.0
3		0.0
4		0.0
...	...	
10025		5.0
10026		0.0
10027		0.0
10028		0.0
10029		0.0

vehicles_PAM_1_OUT_Zona_Andalucia_no_GR		vehicles_PAM_1_OUT_Extranjero \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
10025	1.0	1.0
10026	0.0	0.0
10027	0.0	0.0

10028	0.0	0.0
10029	0.0	0.0

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
10025	1.0
10026	0.0
10027	0.0
10028	0.0
10029	0.0

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
10025	2.0
10026	0.0
10027	0.0
10028	0.0
10029	0.0

	vehicles_PAM_1_OUT_Zona_Otras	vehicles_PAM_1_OUT_5_seats	...	\
0	0.0	0.0	...	
1	0.0	0.0	...	
2	0.0	0.0	...	
3	0.0	0.0	...	
4	0.0	0.0	...	
...	...	...	...	
10025	0.0	16.0	...	
10026	0.0	1.0	...	
10027	0.0	0.0	...	
10028	0.0	0.0	...	
10029	0.0	0.0	...	

	vehicles_BUB_IN_Zona_Otras	vehicles_BUB_IN_5_seats \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0

4	0.0	0.0
...	...	...
10025	0.0	28.0
10026	0.0	21.0
10027	0.0	0.0
10028	0.0	0.0
10029	0.0	0.0

	vehicles_BUB_IN_+5_seats	vehicles_BUB_IN_-5_seats	\
0	0.0	0.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	0.0	0.0	
...	...	...	
10025	6.0	3.0	
10026	2.0	1.0	
10027	0.0	0.0	
10028	0.0	0.0	
10029	0.0	0.0	

	vehicles_BUB_IN_nan_seats	vehicles_BUB_IN_101-200_CO2	\
0	0.0	0.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	0.0	0.0	
...	...	...	
10025	5.0	16.0	
10026	2.0	11.0	
10027	0.0	0.0	
10028	0.0	0.0	
10029	0.0	0.0	

	vehicles_BUB_IN_0-100_CO2	vehicles_BUB_IN_nan_CO2	\
0	0.0	0.0	
1	0.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	0.0	0.0	
...	...	...	
10025	3.0	23.0	
10026	3.0	11.0	
10027	0.0	0.0	
10028	0.0	0.0	
10029	0.0	0.0	

	vehicles_BUB_IN_201-300_CO2	vehicles_BUB_IN_+300_CO2
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
10025	0.0	0.0
10026	1.0	0.0
10027	0.0	0.0
10028	0.0	0.0
10029	0.0	0.0

[10030 rows x 103 columns]

```
[4]: trafico_contamina_interseccion = pd.read_csv('../data/
↳trafico_contamina_intersección.csv')
trafico_contamina_interseccion["Date"] = pd.
↳to_datetime(trafico_contamina_interseccion["Date"])
trafico_contamina_interseccion["Date"] = trafico_contamina_interseccion["Date"].
↳dt.tz_localize("UTC")
trafico_contamina_interseccion.rename(columns={'Date': 'date'}, inplace=True)
```

```
[5]: columns_interseccion = [
    "CO",
    "NO2",
    "NO",
    "O3",
    "PM10",
    "eBC_ff",
    "eBC_bb",
    "TEMP",
    "RH",
    "WS",
    "WD",
    "PRES"
]
# Combinamos ambas listas
columns_interseccion = columns + columns_interseccion

trafico_pam =
↳trafico_contamina_interseccion[trafico_contamina_interseccion["truck_pos"]
↳== "PAM_2"]

df_int = trafico_pam[["date"] + columns_interseccion]
columnas_int64 = df_int.select_dtypes(include='int64').columns
```

```
# Convertir esas columnas a float64
df_int[columnas_int64] = df_int[columnas_int64].astype('float64')

df_int
```

```
[5]:
```

	date	vehicles_PAM_1_OUT \
842	2023-01-17 17:00:00+00:00	6.0
843	2023-01-17 18:00:00+00:00	0.0
844	2023-01-17 19:00:00+00:00	0.0
845	2023-01-17 20:00:00+00:00	0.0
846	2023-01-17 21:00:00+00:00	0.0
...	...	...
2172	2023-06-26 19:00:00+00:00	7.0
2173	2023-06-26 20:00:00+00:00	3.0
2174	2023-06-26 21:00:00+00:00	0.0
2175	2023-06-26 22:00:00+00:00	0.0
2176	2023-06-27 00:00:00+00:00	0.0

	vehicles_PAM_1_OUT_Zona_Granada \
842	4.0
843	0.0
844	0.0
845	0.0
846	0.0
...	...
2172	5.0
2173	3.0
2174	0.0
2175	0.0
2176	0.0

	vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
842	0.0
843	0.0
844	0.0
845	0.0
846	0.0
...	...
2172	1.0
2173	0.0
2174	0.0
2175	0.0
2176	0.0

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR	vehicles_PAM_1_OUT_Extranjero \
842	0.0	1.0
843	0.0	0.0

844	0.0	0.0
845	0.0	0.0
846	0.0	0.0
...	...	...
2172	0.0	1.0
2173	0.0	0.0
2174	0.0	0.0
2175	0.0	0.0
2176	0.0	0.0

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
842	0.0
843	0.0
844	0.0
845	0.0
846	0.0
...	...
2172	0.0
2173	0.0
2174	0.0
2175	0.0
2176	0.0

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
842	1.0
843	0.0
844	0.0
845	0.0
846	0.0
...	...
2172	0.0
2173	0.0
2174	0.0
2175	0.0
2176	0.0

	vehicles_PAM_1_OUT_Zona_Otras	vehicles_PAM_1_OUT_5_seats	...	NO	\
842	0.0	4.0	...	0.8	
843	0.0	0.0	...	0.5	
844	0.0	0.0	...	0.3	
845	0.0	0.0	...	0.2	
846	0.0	0.0	...	0.2	
...	...	...	...	...	
2172	0.0	5.0	...	1.5	
2173	0.0	3.0	...	0.4	
2174	0.0	0.0	...	0.1	
2175	0.0	0.0	...	0.1	

2176					0.0				0.0	...	0.1
	03	PM10	eBC_ff	eBC_bb	TEMP	RH	WS	WD	PRES		
842	NaN	15.4	NaN	NaN	9.8	47.7	NaN	NaN	878.7		
843	NaN	8.8	NaN	NaN	8.6	54.5	NaN	NaN	878.7		
844	NaN	15.7	NaN	NaN	8.3	48.0	NaN	NaN	878.5		
845	NaN	10.2	NaN	NaN	7.8	50.2	NaN	NaN	879.0		
846	NaN	9.9	NaN	NaN	7.4	52.3	NaN	NaN	878.3		
...	...	...	...	...	...	...	...	...	...		
2172	111.4	7.8	0.6	0.1	31.4	21.5	1.12	246.34	893.3		
2173	99.6	23.0	0.3	0.1	28.8	26.0	0.81	7.46	893.6		
2174	78.8	11.3	0.4	0.1	25.7	29.5	1.06	129.23	893.9		
2175	77.4	1.9	0.2	0.1	24.7	27.9	1.25	133.45	894.3		
2176	84.3	14.3	0.1	0.1	24.2	30.3	1.49	123.34	894.5		

[1335 rows x 115 columns]

1.1 Interpolación para completar los valores nulos.

1.2 Rellenar del df_int las horas faltantes con valores NaN

En los datos del camión, poner un registro por cada hora. Si no se encuentra, añadir todos NaN.

El método de pandas `date_range` devuelve el rango de datetimes igualmente espaciados (dada una frecuencia) entre un inicio y un final (por defecto ambos inclusive). El tipo de dato que devuelve es `DatetimeIndex`. Dado que disponemos de dos periodos de fechas con datos disponibles (1er periodo: 17/01/2023-14/03/2023 y 2do periodo 06/06/2023-27/06/2023) realizaremos otro rango para el segundo periodo y lo uniremos mediante el método `union` perteneciente a la clase `Index`.

Explicación paso a paso: - `set_index('date')`: Convierte la columna 'date' en el índice del DataFrame `df_int`. - `reindex(full_date_range)` Rellena el DataFrame para que tenga todas las fechas en `full_date_range`, incluso si algunas no estaban en `df_int`. Esto es útil si hay fechas faltantes y quieres asegurar de que el índice sea continuo. `reset_index()`

Devuelve la columna de fecha como una columna normal en lugar de mantenerla como índice.

```
[6]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Supongamos que el DataFrame inicial es df_int con columnas 'date'

# Definir los dos periodos
start_date_1 = pd.to_datetime("2023-01-17 17:00:00+00:00")
end_date_1 = pd.to_datetime("2023-03-14 11:00:00+00:00")

start_date_2 = pd.to_datetime("2023-06-06 13:00:00+00:00")
end_date_2 = pd.to_datetime("2023-06-27 00:00:00+00:00")
```

```

# Crear un rango completo de fechas con frecuencia horaria para ambos períodos
↳ combinados
full_date_range = pd.date_range(start=start_date_1, end=end_date_1, freq='h').
↳ union(
    pd.date_range(start=start_date_2, end=end_date_2, freq='h')
)

# Asegurar que 'date' sea de tipo datetime
df_int['date'] = pd.to_datetime(df_int['date'])

# Reindexar para completar las horas faltantes
df_int = df_int.set_index('date').reindex(full_date_range).reset_index()
df_int = df_int.rename(columns={'index': 'date'})

df_int

```

```

[6]:
      date  vehicles_PAM_1_OUT \
0  2023-01-17 17:00:00+00:00    6.0
1  2023-01-17 18:00:00+00:00    0.0
2  2023-01-17 19:00:00+00:00    0.0
3  2023-01-17 20:00:00+00:00    0.0
4  2023-01-17 21:00:00+00:00    0.0
...
1826 2023-06-26 20:00:00+00:00    3.0
1827 2023-06-26 21:00:00+00:00    0.0
1828 2023-06-26 22:00:00+00:00    0.0
1829 2023-06-26 23:00:00+00:00    NaN
1830 2023-06-27 00:00:00+00:00    0.0

```

```

      vehicles_PAM_1_OUT_Zona_Granada \
0      4.0
1      0.0
2      0.0
3      0.0
4      0.0
...
1826    3.0
1827    0.0
1828    0.0
1829    NaN
1830    0.0

```

```

      vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
0      0.0
1      0.0
2      0.0
3      0.0

```

4	0.0
...	...
1826	0.0
1827	0.0
1828	0.0
1829	NaN
1830	0.0

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR	vehicles_PAM_1_OUT_Extranjero \
0	0.0	1.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
...	...	...
1826	0.0	0.0
1827	0.0	0.0
1828	0.0	0.0
1829	NaN	NaN
1830	0.0	0.0

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
1826	0.0
1827	0.0
1828	0.0
1829	NaN
1830	0.0

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
0	1.0
1	0.0
2	0.0
3	0.0
4	0.0
...	...
1826	0.0
1827	0.0
1828	0.0
1829	NaN
1830	0.0

	vehicles_PAM_1_OUT_Zona_Otras	vehicles_PAM_1_OUT_5_seats	...	NO	\
0	0.0	4.0	...	0.8	
1	0.0	0.0	...	0.5	
2	0.0	0.0	...	0.3	
3	0.0	0.0	...	0.2	
4	0.0	0.0	...	0.2	
...	...	...	...	...	
1826	0.0	3.0	...	0.4	
1827	0.0	0.0	...	0.1	
1828	0.0	0.0	...	0.1	
1829	NaN	NaN	...	NaN	
1830	0.0	0.0	...	0.1	

	O3	PM10	eBC_ff	eBC_bb	TEMP	RH	WS	WD	PRES
0	NaN	15.4	NaN	NaN	9.8	47.7	NaN	NaN	878.7
1	NaN	8.8	NaN	NaN	8.6	54.5	NaN	NaN	878.7
2	NaN	15.7	NaN	NaN	8.3	48.0	NaN	NaN	878.5
3	NaN	10.2	NaN	NaN	7.8	50.2	NaN	NaN	879.0
4	NaN	9.9	NaN	NaN	7.4	52.3	NaN	NaN	878.3
...	...	...	...	...	...	...	...	...	...
1826	99.6	23.0	0.3	0.1	28.8	26.0	0.81	7.46	893.6
1827	78.8	11.3	0.4	0.1	25.7	29.5	1.06	129.23	893.9
1828	77.4	1.9	0.2	0.1	24.7	27.9	1.25	133.45	894.3
1829	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1830	84.3	14.3	0.1	0.1	24.2	30.3	1.49	123.34	894.5

[1831 rows x 115 columns]

1.3 Período_1

```
[7]: len(df_int.columns)
```

```
[7]: 115
```

```
[8]: df_int["date"] = pd.to_datetime(df_int["date"])
df_int.set_index("date", inplace=True)
```

1.4 División Por Periodos

```
[9]: import pandas as pd

# Definir los periodos
periodo_1 = df_int.loc["2023-01-17":"2023-03-14"]
periodo_2 = df_int.loc["2023-06-06":"2023-06-27"]
```

1.5 División en training, validation y test

Para dividir el periodo del 17 de enero de 2023 al 14 de marzo de 2023 en entrenamiento, validación y prueba siguiendo un esquema análogo al del Beijing Multi-Site Air-Quality Dataset, vamos a asignar los datos de manera similar a cómo se hizo en ese conjunto.

El esquema original divide los datos en tres partes:

- **Fase de entrenamiento (80% del dataset):** Primeros 45 días
 - **Training:** 80% de los 45 días: 36 días
 - **Validación** 20% restante de los 45 días: 9 días.
- **Prueba (20% del dataset):** 20% del dataset: 11 días.

1.5.1 Duración total

El periodo de tiempo es del 17 de enero de 2023 al 14 de marzo de 2023. **hay 57 días.**

1.6 Resumen:

- **Conjunto de entrenamiento:** 17 de enero de 2023 - 21 de febrero de 2023 (36 días)
- **Conjunto de validación:** 22 de febrero de 2023 - 2 de marzo de 2023 (9 días).
- **Conjunto de prueba:** 3 de marzo de 2023 - 14 de marzo de 2023) (12 días)

```
[10]: periodo_1.loc["2023-03-03":"2023-03-14"]
```

```
[10]: vehicles_PAM_1_OUT \
```

```
date
2023-03-03 00:00:00+00:00      0.0
2023-03-03 01:00:00+00:00     NaN
2023-03-03 02:00:00+00:00     NaN
2023-03-03 03:00:00+00:00     NaN
2023-03-03 04:00:00+00:00     NaN
...
2023-03-14 07:00:00+00:00     NaN
2023-03-14 08:00:00+00:00      2.0
2023-03-14 09:00:00+00:00      6.0
2023-03-14 10:00:00+00:00     10.0
2023-03-14 11:00:00+00:00      3.0
```

```
vehicles_PAM_1_OUT_Zona_Granada \
```

```
date
2023-03-03 00:00:00+00:00      0.0
2023-03-03 01:00:00+00:00     NaN
2023-03-03 02:00:00+00:00     NaN
2023-03-03 03:00:00+00:00     NaN
2023-03-03 04:00:00+00:00     NaN
...
2023-03-14 07:00:00+00:00     NaN
2023-03-14 08:00:00+00:00      1.0
2023-03-14 09:00:00+00:00      4.0
```

2023-03-14 10:00:00+00:00	6.0
2023-03-14 11:00:00+00:00	3.0

vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \	
date	
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 07:00:00+00:00	NaN
2023-03-14 08:00:00+00:00	0.0
2023-03-14 09:00:00+00:00	0.0
2023-03-14 10:00:00+00:00	1.0
2023-03-14 11:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Andalucia_no_GR \	
date	
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 07:00:00+00:00	NaN
2023-03-14 08:00:00+00:00	0.0
2023-03-14 09:00:00+00:00	1.0
2023-03-14 10:00:00+00:00	1.0
2023-03-14 11:00:00+00:00	0.0

vehicles_PAM_1_OUT_Extranjero \	
date	
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 07:00:00+00:00	NaN
2023-03-14 08:00:00+00:00	1.0
2023-03-14 09:00:00+00:00	1.0
2023-03-14 10:00:00+00:00	0.0
2023-03-14 11:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \	
date	

2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 07:00:00+00:00	NaN
2023-03-14 08:00:00+00:00	0.0
2023-03-14 09:00:00+00:00	0.0
2023-03-14 10:00:00+00:00	1.0
2023-03-14 11:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \

date	
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 07:00:00+00:00	NaN
2023-03-14 08:00:00+00:00	0.0
2023-03-14 09:00:00+00:00	0.0
2023-03-14 10:00:00+00:00	0.0
2023-03-14 11:00:00+00:00	0.0

vehicles_PAM_1_OUT_Zona_Otras \

date	
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 07:00:00+00:00	NaN
2023-03-14 08:00:00+00:00	0.0
2023-03-14 09:00:00+00:00	0.0
2023-03-14 10:00:00+00:00	1.0
2023-03-14 11:00:00+00:00	0.0

vehicles_PAM_1_OUT_5_seats \

date	
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN


```

...
2023-03-14 07:00:00+00:00      NaN
2023-03-14 08:00:00+00:00      0.0
2023-03-14 09:00:00+00:00      5.0
2023-03-14 10:00:00+00:00      8.0
2023-03-14 11:00:00+00:00      2.0

vehicles_PAM_1_OUT_+5_seats ... NO    03  PM10  \
date
2023-03-03 00:00:00+00:00      0.0 ... 0.1  74.6  6.5
2023-03-03 01:00:00+00:00      NaN ... NaN   NaN   NaN
2023-03-03 02:00:00+00:00      NaN ... NaN   NaN   NaN
2023-03-03 03:00:00+00:00      NaN ... NaN   NaN   NaN
2023-03-03 04:00:00+00:00      NaN ... NaN   NaN   NaN
...
2023-03-14 07:00:00+00:00      NaN ... NaN   NaN   NaN
2023-03-14 08:00:00+00:00      1.0 ... 0.6  53.4   NaN
2023-03-14 09:00:00+00:00      0.0 ... 2.0  51.4   NaN
2023-03-14 10:00:00+00:00      2.0 ... 1.2  57.9   NaN
2023-03-14 11:00:00+00:00      0.0 ... 3.1  66.0   NaN

eBC_ff  eBC_bb  TEMP    RH  WS  WD    PRES
date
2023-03-03 00:00:00+00:00      0.2    0.1    0.4  57.3 NaN NaN  888.1
2023-03-03 01:00:00+00:00      NaN    NaN    NaN   NaN NaN NaN   NaN
2023-03-03 02:00:00+00:00      NaN    NaN    NaN   NaN NaN NaN   NaN
2023-03-03 03:00:00+00:00      NaN    NaN    NaN   NaN NaN NaN   NaN
2023-03-03 04:00:00+00:00      NaN    NaN    NaN   NaN NaN NaN   NaN
...
2023-03-14 07:00:00+00:00      NaN    NaN    NaN   NaN NaN NaN   NaN
2023-03-14 08:00:00+00:00      NaN    NaN    8.7  58.3 NaN NaN  889.1
2023-03-14 09:00:00+00:00      NaN    NaN    9.3  59.1 NaN NaN  889.6
2023-03-14 10:00:00+00:00      NaN    NaN   10.6  56.6 NaN NaN  890.6
2023-03-14 11:00:00+00:00      NaN    NaN   14.4  56.7 NaN NaN  891.0

```

[276 rows x 114 columns]

```

[11]: # Crear el índice con las fechas y horas de 2023-01-17 00:00:00 hasta
      ↪ 2023-01-17 17:00:00
date_range = pd.date_range(start="2023-01-17 00:00:00", end="2023-01-17 16:00:
      ↪ 00", freq="H", tz="UTC")

# Crear el índice con las fechas y horas de 2023-01-17 12:00:00 hasta
      ↪ 2023-01-17 23:00:00
date_range_end = pd.date_range(start="2023-03-14 12:00:00", end="2023-03-14 23:
      ↪ 00:00", freq="H", tz="UTC")

```

```
# Definir las columnas de calidad del aire
columns = ["vehicles_PAM_1_OUT", "CO", "NO2", "NO", "O3", "PM10", "eBC_ff",
↪ "eBC_bb", "TEMP", "RH", "WS", "WD", "PRES"]

# Crear un DataFrame con el índice y columnas, relleno los valores con NaN
df = pd.DataFrame(np.nan, index=date_range, columns=columns)
df_end = pd.DataFrame(np.nan, index=date_range_end, columns=columns)
periodo_1_df = pd.concat([df, periodo_1, df_end], axis=0) # axis=0 indica
↪ concatenar por filas
```

```
/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_98675/3814031928.py:2
: FutureWarning: 'H' is deprecated and will be removed in a future version,
please use 'h' instead.
```

```
date_range = pd.date_range(start="2023-01-17 00:00:00", end="2023-01-17
16:00:00", freq="H", tz="UTC")
```

```
/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_98675/3814031928.py:5
: FutureWarning: 'H' is deprecated and will be removed in a future version,
please use 'h' instead.
```

```
date_range_end = pd.date_range(start="2023-03-14 12:00:00", end="2023-03-14
23:00:00", freq="H", tz="UTC")
```

[12]: periodo_1_df

```
[12]:
```

	vehicles_PAM_1_OUT	CO	NO2	NO	O3	PM10	eBC_ff	\
2023-01-17 00:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
...	...	...	...	...	...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

	eBC_bb	TEMP	RH	...	vehicles_BUB_IN_Zona_Otras	\
2023-01-17 00:00:00+00:00	NaN	NaN	NaN	...		NaN
2023-01-17 01:00:00+00:00	NaN	NaN	NaN	...		NaN
2023-01-17 02:00:00+00:00	NaN	NaN	NaN	...		NaN
2023-01-17 03:00:00+00:00	NaN	NaN	NaN	...		NaN
2023-01-17 04:00:00+00:00	NaN	NaN	NaN	...		NaN
...	...	...	...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	NaN	...		NaN
2023-03-14 20:00:00+00:00	NaN	NaN	NaN	...		NaN
2023-03-14 21:00:00+00:00	NaN	NaN	NaN	...		NaN
2023-03-14 22:00:00+00:00	NaN	NaN	NaN	...		NaN

2023-03-14 23:00:00+00:00	NaN	NaN	NaN	...	NaN
---------------------------	-----	-----	-----	-----	-----

	vehicles_BUB_IN_5_seats	vehicles_BUB_IN_+5_seats	\
2023-01-17 00:00:00+00:00	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	
...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	

	vehicles_BUB_IN_-5_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_nan_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_101-200_CO2	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	

```

...
2023-03-14 19:00:00+00:00      NaN
2023-03-14 20:00:00+00:00      NaN
2023-03-14 21:00:00+00:00      NaN
2023-03-14 22:00:00+00:00      NaN
2023-03-14 23:00:00+00:00      NaN

vehicles_BUB_IN_0-100_CO2  vehicles_BUB_IN_nan_CO2  \
2023-01-17 00:00:00+00:00      NaN      NaN
2023-01-17 01:00:00+00:00      NaN      NaN
2023-01-17 02:00:00+00:00      NaN      NaN
2023-01-17 03:00:00+00:00      NaN      NaN
2023-01-17 04:00:00+00:00      NaN      NaN
...
2023-03-14 19:00:00+00:00      NaN      NaN
2023-03-14 20:00:00+00:00      NaN      NaN
2023-03-14 21:00:00+00:00      NaN      NaN
2023-03-14 22:00:00+00:00      NaN      NaN
2023-03-14 23:00:00+00:00      NaN      NaN

vehicles_BUB_IN_201-300_CO2  \
2023-01-17 00:00:00+00:00      NaN
2023-01-17 01:00:00+00:00      NaN
2023-01-17 02:00:00+00:00      NaN
2023-01-17 03:00:00+00:00      NaN
2023-01-17 04:00:00+00:00      NaN
...
2023-03-14 19:00:00+00:00      NaN
2023-03-14 20:00:00+00:00      NaN
2023-03-14 21:00:00+00:00      NaN
2023-03-14 22:00:00+00:00      NaN
2023-03-14 23:00:00+00:00      NaN

vehicles_BUB_IN_+300_CO2
2023-01-17 00:00:00+00:00      NaN
2023-01-17 01:00:00+00:00      NaN
2023-01-17 02:00:00+00:00      NaN
2023-01-17 03:00:00+00:00      NaN
2023-01-17 04:00:00+00:00      NaN
...
2023-03-14 19:00:00+00:00      NaN
2023-03-14 20:00:00+00:00      NaN
2023-03-14 21:00:00+00:00      NaN
2023-03-14 22:00:00+00:00      NaN
2023-03-14 23:00:00+00:00      NaN

```

[1368 rows x 114 columns]

```
[13]: periodo_1_df.loc["2023-03-03":"2023-03-14"]
```

```
[13]:
```

	vehicles_PAM_1_OUT	CO	NO2	NO	O3	PM10	\
2023-03-03 00:00:00+00:00	0.0	0.21	3.6	0.1	74.6	6.5	
2023-03-03 01:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-03 02:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-03 03:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-03 04:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
...	...	...	...	...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	

	eBC_ff	eBC_bb	TEMP	RH	...	\
2023-03-03 00:00:00+00:00	0.2	0.1	0.4	57.3	...	
2023-03-03 01:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-03-03 02:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-03-03 03:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-03-03 04:00:00+00:00	NaN	NaN	NaN	NaN	...	
...	...	...	...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-03-14 20:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-03-14 21:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-03-14 22:00:00+00:00	NaN	NaN	NaN	NaN	...	
2023-03-14 23:00:00+00:00	NaN	NaN	NaN	NaN	...	

	vehicles_BUB_IN_Zona_Otras	\
2023-03-03 00:00:00+00:00	0.0	
2023-03-03 01:00:00+00:00	NaN	
2023-03-03 02:00:00+00:00	NaN	
2023-03-03 03:00:00+00:00	NaN	
2023-03-03 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_5_seats	vehicles_BUB_IN_+5_seats	\
2023-03-03 00:00:00+00:00	0.0	0.0	
2023-03-03 01:00:00+00:00	NaN	NaN	
2023-03-03 02:00:00+00:00	NaN	NaN	
2023-03-03 03:00:00+00:00	NaN	NaN	
2023-03-03 04:00:00+00:00	NaN	NaN	

...	...	...
2023-03-14 19:00:00+00:00	NaN	NaN
2023-03-14 20:00:00+00:00	NaN	NaN
2023-03-14 21:00:00+00:00	NaN	NaN
2023-03-14 22:00:00+00:00	NaN	NaN
2023-03-14 23:00:00+00:00	NaN	NaN

	vehicles_BUB_IN_-5_seats \
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN

...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

	vehicles_BUB_IN_nan_seats \
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN

...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

	vehicles_BUB_IN_101-200_CO2 \
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN

...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

vehicles_BUB_IN_0-100_CO2	vehicles_BUB_IN_nan_CO2 \
---------------------------	---------------------------

2023-03-03 00:00:00+00:00	0.0	0.0
2023-03-03 01:00:00+00:00	NaN	NaN
2023-03-03 02:00:00+00:00	NaN	NaN
2023-03-03 03:00:00+00:00	NaN	NaN
2023-03-03 04:00:00+00:00	NaN	NaN
...	...	...
2023-03-14 19:00:00+00:00	NaN	NaN
2023-03-14 20:00:00+00:00	NaN	NaN
2023-03-14 21:00:00+00:00	NaN	NaN
2023-03-14 22:00:00+00:00	NaN	NaN
2023-03-14 23:00:00+00:00	NaN	NaN

	vehicles_BUB_IN_201-300_CO2 \
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

	vehicles_BUB_IN_+300_CO2
2023-03-03 00:00:00+00:00	0.0
2023-03-03 01:00:00+00:00	NaN
2023-03-03 02:00:00+00:00	NaN
2023-03-03 03:00:00+00:00	NaN
2023-03-03 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

[288 rows x 114 columns]

```
[14]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from typing import Dict

def sliding_window(data, n_steps):
    X = []
```

```

for i in range(0, len(data), n_steps):
    # Solo agregamos ventanas completas
    if i + n_steps <= len(data):
        X.append(data[i:i + n_steps])

# Convertimos la lista de ventanas a un array
return np.array(X)

def create_missingness(data: np.ndarray, rate: float, pattern: str, **kwargs) → np.ndarray:
    """
    Introduce valores faltantes en el conjunto de datos con un patrón
    específico.
    """
    n_samples, n_features, n_steps = data.shape
    missing_data = data.copy()

    if pattern == "point":
        # Introducir valores faltantes de forma aleatoria (punto a punto)
        for i in range(n_samples):
            for j in range(n_features):
                if np.random.rand() < rate:
                    missing_data[i, j, np.random.randint(n_steps)] = np.nan
    elif pattern == "subseq":
        # Introducir valores faltantes en subsecuencias
        for i in range(n_samples):
            for j in range(n_features):
                if np.random.rand() < rate:
                    start_idx = np.random.randint(n_steps)
                    length = np.random.randint(1, n_steps - start_idx)
                    missing_data[i, j, start_idx:start_idx + length] = np.nan
    elif pattern == "block":
        # Introducir valores faltantes en bloques
        for i in range(n_samples):
            for j in range(n_features):
                if np.random.rand() < rate:
                    block_size = np.random.randint(1, n_steps)
                    missing_data[i, j, :block_size] = np.nan
    return missing_data

```

```

[15]: # Seleccionar solo las columnas necesarias para las características
features = columns_interseccion

```

```

[16]: import pandas as pd
from sklearn.preprocessing import StandardScaler

```



```

def preprocess_periodo_1_data(
    periodo_1,
    rate: float,
    n_steps: int,
    pattern: str = "point",
    **kwargs,
) -> dict:

    # Asegúrate de que el índice sea de tipo datetime
    if not pd.api.types.is_datetime64_any_dtype(periodo_1.index):
        periodo_1.index = pd.to_datetime(periodo_1.index)

    # Verificar las fechas en el DataFrame
    print("Fechas en periodo_1:", periodo_1.index.min(), "a", periodo_1.index.
    ↪max())

    # Conjunto de entrenamiento: 17 enero - 21 febrero 2023 (36 días)
    mask_train = (periodo_1.index >= "2023-01-17") & (periodo_1.index <
    ↪"2023-02-22")
    train_set = periodo_1[mask_train]

    # Verificar si hay datos en train_set
    print("Datos en conjunto de entrenamiento:", train_set.shape)

    # Conjunto de validación: 22 febrero - 2 marzo 2023 (9 días)
    mask_val = (periodo_1.index >= "2023-02-22") & (periodo_1.index <
    ↪"2023-03-03")
    val_set = periodo_1[mask_val]

    # Verificar si hay datos en val_set
    print("Datos en conjunto de validación:", val_set.shape)

    # Conjunto de prueba: 3 marzo - 14 marzo 2023 (11 días)
    mask_test = (periodo_1.index >= "2023-03-03") & (periodo_1.index <
    ↪"2023-03-15")
    test_set = periodo_1[mask_test]

    # Verificar si hay datos en test_set
    print("Datos en conjunto de prueba:", test_set.shape)

    # Verificar si los conjuntos de datos no están vacíos antes de proceder
    if test_set.empty or val_set.empty or train_set.empty:
        raise ValueError("Uno o más conjuntos de datos están vacíos. Revisa las
    ↪fechas y los filtros.")

    train_set = train_set[features]

```

```

val_set = val_set[features]
test_set = test_set[features]

# Normalización de datos
scaler = StandardScaler()
train_X = scaler.fit_transform(train_set.loc[:, features])
val_X = scaler.transform(val_set.loc[:, features])
test_X = scaler.transform(test_set.loc[:, features])

# Crear ventanas deslizantes
train_X = sliding_window(train_X, n_steps)
val_X = sliding_window(val_X, n_steps)
test_X = sliding_window(test_X, n_steps)

# Ensamblar el conjunto de datos final
processed_dataset = {
    "n_steps": n_steps,
    "n_features": train_X.shape[-1], # Número de características
    "scaler": scaler,
    "train_X": train_X,
    "val_X": val_X,
    "test_X": test_X,
    "train_X_ori": train_X, # Versión original de los datos de
    ↪entrenamiento
    "val_X_ori": val_X, # Versión original de los datos de validación
    "test_X_ori": test_X, # Versión original de los datos de prueba
}

if rate > 0:
    # hold out ground truth in the original data for evaluation
    train_X_ori = train_X
    val_X_ori = val_X
    test_X_ori = test_X

    # mask values in the train set to keep the same with below validation
    ↪and test sets
    train_X = create_missingness(train_X, rate, pattern, **kwargs)
    # mask values in the validation set as ground truth
    val_X = create_missingness(val_X, rate, pattern, **kwargs)
    # mask values in the test set as ground truth
    test_X = create_missingness(test_X, rate, pattern, **kwargs)

    # Revertir el escalado (transformación inversa)
    train_X_not_scale = scaler.inverse_transform(train_X.reshape(-1,
    ↪train_X.shape[-1])) # Ajuste de forma si es necesario

```

```

        val_X_not_scale = scaler.inverse_transform(val_X.reshape(-1, val_X.
↪shape[-1])) # Ajuste de forma si es necesario
        test_X_not_scale = scaler.inverse_transform(test_X.reshape(-1, test_X.
↪shape[-1])) # Ajuste de forma si es necesario

        processed_dataset["train_X"] = train_X
        processed_dataset["train_X_not_scale"] = train_X_not_scale
        processed_dataset["train_X_ori"] = train_X_ori

        processed_dataset["val_X"] = val_X
        processed_dataset["val_X_not_scale"] = val_X_not_scale
        processed_dataset["val_X_ori"] = val_X_ori

        processed_dataset["test_X"] = test_X
        processed_dataset["test_X_not_scale"] = test_X_not_scale
        processed_dataset["test_X_ori"] = test_X_ori
    else:
        logger.warning("rate is 0, no missing values are artificially added.")

    return processed_dataset

```

[17]: periodo_1_df

```

[17]:
vehicles_PAM_1_OUT  CO  NO2  NO  O3  PM10  eBC_ff  \
2023-01-17 00:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 01:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 02:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 03:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-01-17 04:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
...
2023-03-14 19:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 20:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 21:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 22:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN
2023-03-14 23:00:00+00:00    NaN NaN  NaN NaN NaN    NaN    NaN

eBC_bb  TEMP  RH  ...  vehicles_BUB_IN_Zona_Otras  \
2023-01-17 00:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 01:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 02:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 03:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-01-17 04:00:00+00:00    NaN    NaN NaN  ...    NaN
...
2023-03-14 19:00:00+00:00    NaN    NaN NaN  ...    NaN
2023-03-14 20:00:00+00:00    NaN    NaN NaN  ...    NaN

```

2023-03-14 21:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 22:00:00+00:00	NaN	NaN	NaN	...	NaN
2023-03-14 23:00:00+00:00	NaN	NaN	NaN	...	NaN

	vehicles_BUB_IN_5_seats	vehicles_BUB_IN_+5_seats	\
2023-01-17 00:00:00+00:00	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	
...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	

	vehicles_BUB_IN_-5_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_nan_seats	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_101-200_CO2	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	

2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

	vehicles_BUB_IN_0-100_CO2	vehicles_BUB_IN_nan_CO2	\
2023-01-17 00:00:00+00:00	NaN	NaN	
2023-01-17 01:00:00+00:00	NaN	NaN	
2023-01-17 02:00:00+00:00	NaN	NaN	
2023-01-17 03:00:00+00:00	NaN	NaN	
2023-01-17 04:00:00+00:00	NaN	NaN	
...	...	...	
2023-03-14 19:00:00+00:00	NaN	NaN	
2023-03-14 20:00:00+00:00	NaN	NaN	
2023-03-14 21:00:00+00:00	NaN	NaN	
2023-03-14 22:00:00+00:00	NaN	NaN	
2023-03-14 23:00:00+00:00	NaN	NaN	

	vehicles_BUB_IN_201-300_CO2	\
2023-01-17 00:00:00+00:00	NaN	
2023-01-17 01:00:00+00:00	NaN	
2023-01-17 02:00:00+00:00	NaN	
2023-01-17 03:00:00+00:00	NaN	
2023-01-17 04:00:00+00:00	NaN	
...	...	
2023-03-14 19:00:00+00:00	NaN	
2023-03-14 20:00:00+00:00	NaN	
2023-03-14 21:00:00+00:00	NaN	
2023-03-14 22:00:00+00:00	NaN	
2023-03-14 23:00:00+00:00	NaN	

	vehicles_BUB_IN_+300_CO2
2023-01-17 00:00:00+00:00	NaN
2023-01-17 01:00:00+00:00	NaN
2023-01-17 02:00:00+00:00	NaN
2023-01-17 03:00:00+00:00	NaN
2023-01-17 04:00:00+00:00	NaN
...	...
2023-03-14 19:00:00+00:00	NaN
2023-03-14 20:00:00+00:00	NaN
2023-03-14 21:00:00+00:00	NaN
2023-03-14 22:00:00+00:00	NaN
2023-03-14 23:00:00+00:00	NaN

[1368 rows x 114 columns]

```
[18]: print(periodo_1_df.columns)
```

```
Index(['vehicles_PAM_1_OUT', 'CO', 'NO2', 'NO', 'O3', 'PM10', 'eBC_ff',  
      'eBC_bb', 'TEMP', 'RH',  
      ...  
      'vehicles_BUB_IN_Zona_Otras', 'vehicles_BUB_IN_5_seats',  
      'vehicles_BUB_IN_+5_seats', 'vehicles_BUB_IN_-5_seats',  
      'vehicles_BUB_IN_nan_seats', 'vehicles_BUB_IN_101-200_CO2',  
      'vehicles_BUB_IN_0-100_CO2', 'vehicles_BUB_IN_nan_CO2',  
      'vehicles_BUB_IN_201-300_CO2', 'vehicles_BUB_IN_+300_CO2'],  
      dtype='object', length=114)
```

```
[19]: # Procesar los datos con una tasa de valores faltantes del 10% y usando el  
      ↪ patrón 'point'  
      periodo_1_dataset = preprocess_periodo_1_data(periodo_1_df, n_steps=24, rate=0.  
      ↪ 1, pattern='point')  
  
      dataset_for_training = {  
          "X": periodo_1_dataset['train_X'],  
      }  
  
      dataset_for_validating = {  
          "X": periodo_1_dataset['val_X'],  
          "X_ori": periodo_1_dataset['val_X_ori'],  
      }  
  
      dataset_for_testing = {  
          "X": periodo_1_dataset['test_X'],  
          "X_ori": periodo_1_dataset['test_X_ori']  
      }
```

Fechas en periodo_1: 2023-01-17 00:00:00+00:00 a 2023-03-14 23:00:00+00:00

Datos en conjunto de entrenamiento: (864, 114)

Datos en conjunto de validación: (216, 114)

Datos en conjunto de prueba: (288, 114)

```
[20]: dataset_for_testing['X']
```

```
[20]: array([[[-0.63318232, -0.65958335, -0.42120251, ...,      nan,  
              nan, -0.75252133],  
            [      nan,      nan,      nan, ...,      nan,  
              nan,      nan],  
            [      nan,      nan,      nan, ...,      nan,  
              nan,      nan],  
            ...,  
            ...])
```

```

[-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -1.00506973],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.89984123],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.89984123]],

[[-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.89984123],
 [ nan, nan, nan, ..., nan,
  nan, nan],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.85774983],
 ...,
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.75252133],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.64729283],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.66833853]],

[[-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.75252133],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.81565843],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -0.81565843],
 ...,
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -1.57330363],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -1.69957783],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, -1.74166923]],

...,

[[-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, 0.13139807],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, 0.13139807],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, 0.02616957],
 ...,
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,
  nan, 0.06826097],
 [-0.63318232, -0.65958335, -0.42120251, ..., nan,

```

```

nan, -0.03696753],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
nan, 0.00512387]],

[[-0.63318232, -0.65958335, -0.42120251, ..., nan,
nan, -0.14219603],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
nan, -0.14219603],
[ nan, nan, nan, ..., nan,
nan, nan],
...,
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan]],

[[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
...,
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan],
[ nan, nan, nan, ..., nan,
nan, nan]], shape=(12, 24, 114))

```

1.7 Imputación baseline: mediante la mediana

```

[21]: import numpy as np

# Copiar el array original
X_imputed_ori = dataset_for_testing['X'].copy()

# Calcular la mediana de cada columna ignorando los NaN
column_medians = np.nanmedian(X_imputed_ori, axis=0)

# Calcular la media de cada fila ignorando los NaN
row_means = np.nanmean(X_imputed_ori, axis=1)

# Reemplazar las columnas que son completamente NaN con la media de las filas
# Usamos la media de las filas para las columnas que tienen NaN en la mediana

```



```

column_medians[np.isnan(column_medians)] = np.nanmean(row_means)

# Evitar impresión completa de grandes arrays
np.set_printoptions(threshold=10)

# Reemplazar los NaN en cada columna con su respectiva mediana
X_imputed = np.where(np.isnan(X_imputed_ori), column_medians, X_imputed_ori)

# Cambiar la configuración de numpy para mostrar el array completo
np.set_printoptions(threshold=np.inf) # Mostrar todos los elementos

# Limitar la impresión a los primeros 100 elementos, por ejemplo
np.set_printoptions(threshold=100)

# Imprimir las medianas calculadas
X_imputed

```

```

/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_98675/1958606079.py:7
: RuntimeWarning: All-NaN slice encountered
  column_medians = np.nanmedian(X_imputed_ori, axis=0)
/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_98675/1958606079.py:1
0: RuntimeWarning: Mean of empty slice
  row_means = np.nanmean(X_imputed_ori, axis=1)

```

```

[21]: array([[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.75252133],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.14219603],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.81565843],
              ...,
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -1.00506973],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.89984123],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.89984123]],

              [[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.89984123],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.14219603],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.85774983],
              ...,
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
              0.04802526, -0.75252133],

```

```

[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.64729283],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.66833853]],

[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.75252133],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.81565843],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.81565843],
...,
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -1.57330363],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -1.69957783],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -1.74166923]],

...,

[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, 0.13139807],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, 0.13139807],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, 0.02616957],
...,
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, 0.06826097],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.03696753],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, 0.00512387]],

[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.14219603],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.14219603],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.81565843],
...,
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.75252133],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
 0.04802526, -0.75252133],
[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,

```

```

0.04802526, -0.66833853]],

[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.60520143],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.14219603],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.81565843],
 ...,
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.75252133],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.75252133],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.66833853]]], shape=(12, 24, 114))

```

1.8 Imputación por la media

```

[22]: import numpy as np

# Copiar el array original
X_imputed_ori = dataset_for_testing['X'].copy()

# Calcular la media de cada columna ignorando los NaN
column_means = np.nanmean(X_imputed_ori, axis=0)

# Calcular la media de cada fila ignorando los NaN
row_means = np.nanmean(X_imputed_ori, axis=1)

# Reemplazar las columnas que son completamente NaN con la media de las filas
# Usamos la media de las filas para las columnas que tienen NaN en la media
column_means[np.isnan(column_means)] = np.nanmean(row_means)

# Evitar impresión completa de grandes arrays
np.set_printoptions(threshold=10)

# Reemplazar los NaN en cada columna con su respectiva media
X_imputed_mean = np.where(np.isnan(X_imputed_ori), column_means, X_imputed_ori)

# Cambiar la configuración de numpy para mostrar el array completo
np.set_printoptions(threshold=np.inf) # Mostrar todos los elementos

# Limitar la impresión a los primeros 100 elementos, por ejemplo
np.set_printoptions(threshold=100)

# Imprimir las medias calculadas
X_imputed_mean

```

```

/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_98675/1551708174.py:7
: RuntimeWarning: Mean of empty slice
  column_means = np.nanmean(X_imputed_ori, axis=0)
/var/folders/gn/9s3465fs47zbr19dkkb4dxwh0000gn/T/ipykernel_98675/1551708174.py:1
0: RuntimeWarning: Mean of empty slice
  row_means = np.nanmean(X_imputed_ori, axis=1)

```

```

[22]: array([[[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.75252133],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.36107131],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.54907956],
              ...,
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -1.00506973],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.89984123],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.89984123]],

             [[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.89984123],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.36107131],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.85774983],
              ...,
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.75252133],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.64729283],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.66833853]],

             [[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.75252133],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.81565843],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -0.81565843],
              ...,
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -1.57330363],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -1.69957783],
              [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
               0.04802526, -1.69957783]]]])

```

```

0.04802526, -1.74166923]],
...,
[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, 0.13139807],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, 0.13139807],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, 0.02616957],
 ...,
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, 0.06826097],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.03696753],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, 0.00512387]],
[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.14219603],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.14219603],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.54907956],
 ...,
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.65664647],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.65430806],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.50899251]],
[[-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.49734222],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.36107131],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.54907956],
 ...,
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.65664647],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.65430806],
 [-0.63318232, -0.65958335, -0.42120251, ..., 0.04802526,
  0.04802526, -0.50899251]]], shape=(12, 24, 114))

```

1.9 Graficar el primer periodo de las tres variables de contaminación

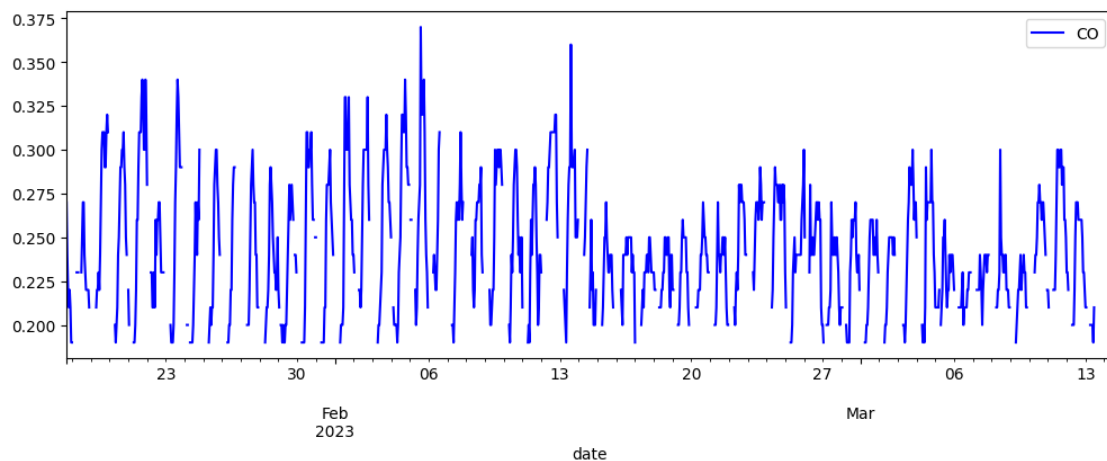
```
[23]: import matplotlib.pyplot as plt
import warnings

# Configurar el tamaño global de la figura
plt.rcParams["figure.figsize"] = [12,4]

# Graficar con un color personalizado
periodo_1[["CO"]].plot(color='blue')

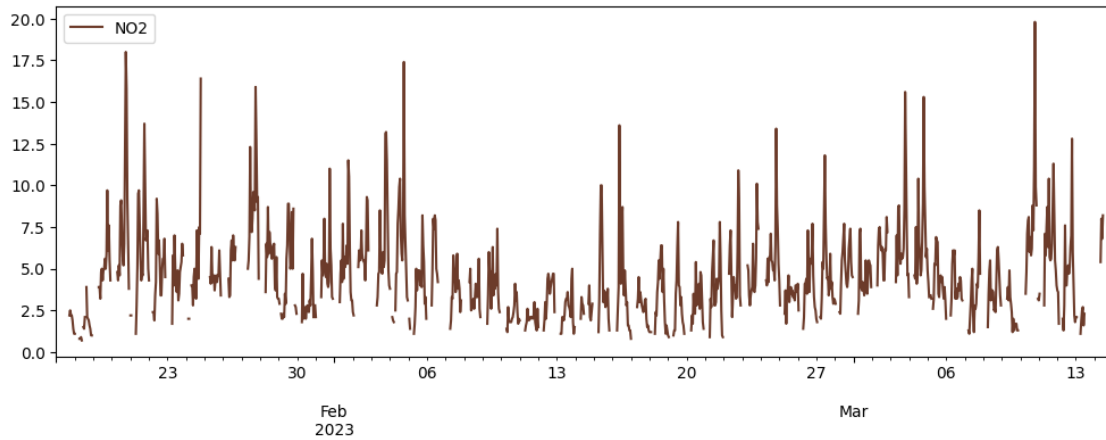
warnings.filterwarnings("ignore")

# Mostrar el gráfico
plt.show()
```



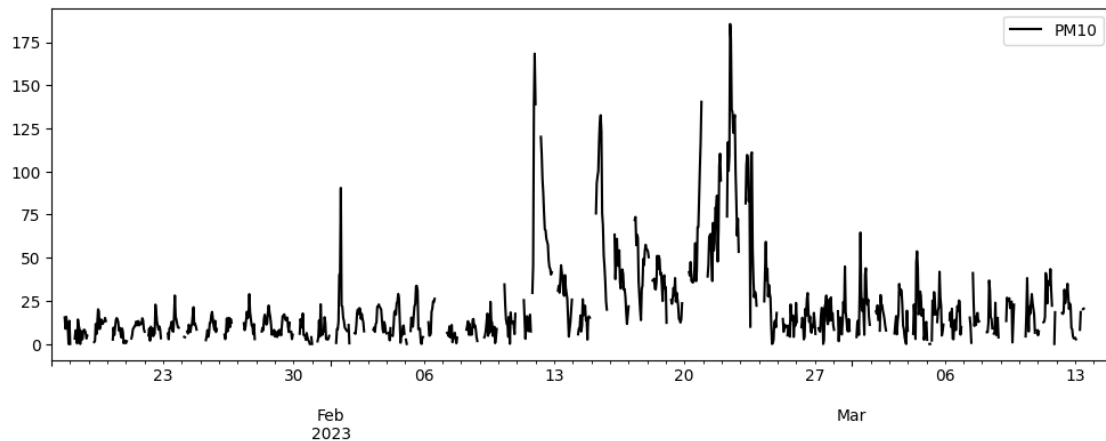
```
[24]: periodo_1_df[["NO2"]].plot(color='#6c3b2a')
```

```
[24]: <Axes: >
```



```
[25]: periodo_1_df[["PM10"]].plot(color='black')
```

```
[25]: <Axes: >
```



1.10 Graficar el segundo periodo de las tres variables de contaminación

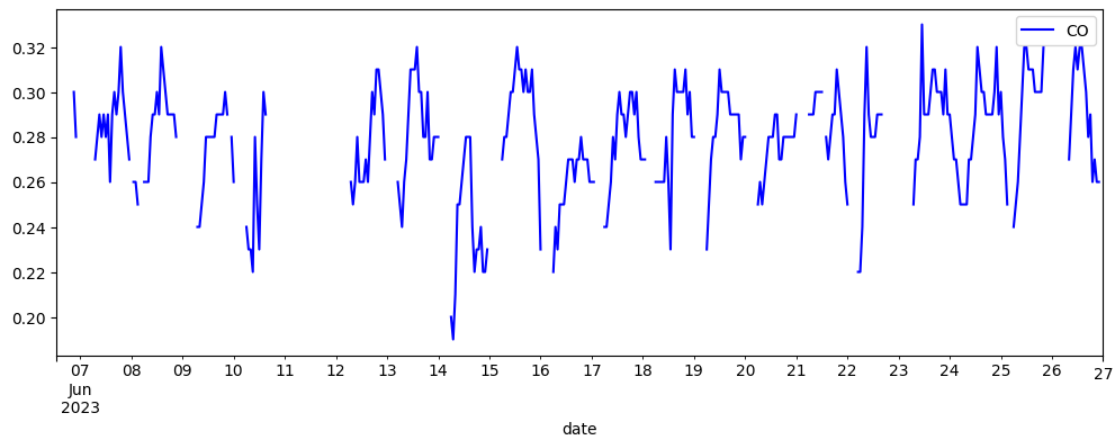
```
[26]: import matplotlib.pyplot as plt

# Configurar el tamaño global de la figura
plt.rcParams["figure.figsize"] = [12,4]

# Graficar con un color personalizado
periodo_2[["CO"]].plot(color='blue')

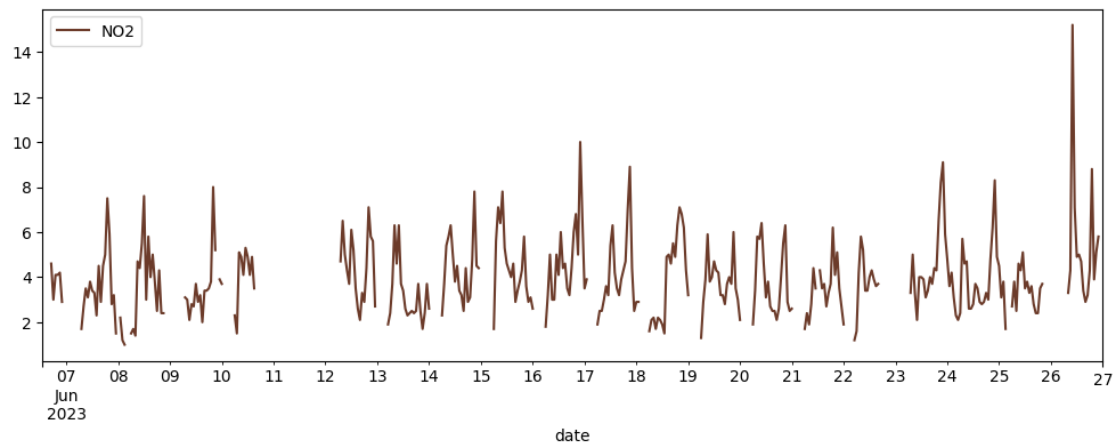
# Mostrar el gráfico
```

```
plt.show()
```



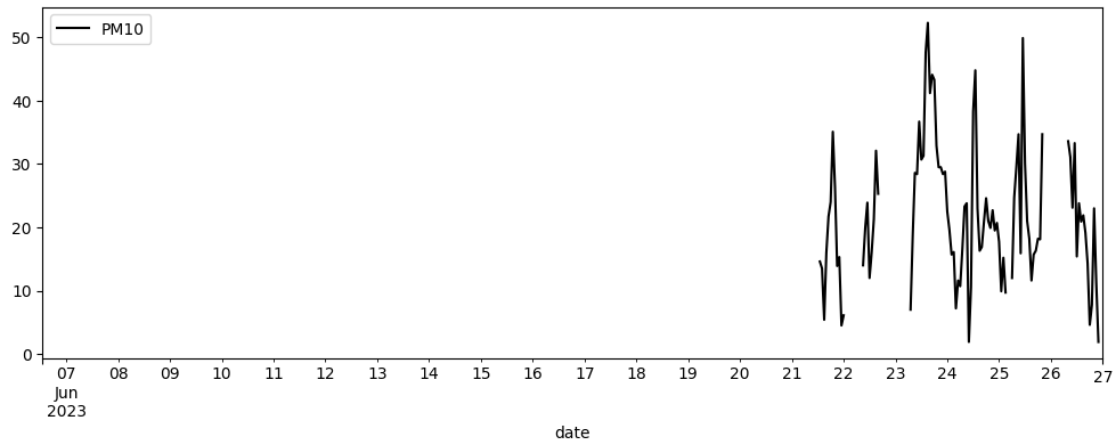
```
[27]: periodo_2[["NO2"]].plot(color='#6c3b2a')
```

```
[27]: <Axes: xlabel='date'>
```



```
[28]: periodo_2[["PM10"]].plot(color='black')
```

```
[28]: <Axes: xlabel='date'>
```

```
[29]: dataset_for_validating
```

```
[29]: {'X': array([[ nan,      nan,      nan, ...,      nan,
                    nan,      nan],
                  [ nan,      nan,      nan, ...,      nan,
                    nan,      nan],
                  [ nan,      nan,      nan, ...,      nan,
                    nan,      nan],
                  ...,
                  [-0.63318232, -0.65958335, -0.42120251, ...,      nan,
                    nan, -1.25761813],
                  [-0.63318232, -0.65958335, -0.42120251, ...,      nan,
                    nan, -1.13134393],
                  [ nan,      nan,      nan, ...,      nan,
                    nan,      nan]],
        [[ nan,      nan,      nan, ...,      nan,
            nan,      nan],
          [ nan,      nan,      nan, ...,      nan,
            nan,      nan],
          [ nan,      nan,      nan, ...,      nan,
            nan,      nan],
          ...,
          [-0.63318232, -0.65958335, -0.42120251, ...,      nan,
            nan, -2.60454293],
          [ nan,      nan,      nan, ...,      nan,
            nan,      nan],
          [ nan,      nan,      nan, ...,      nan,
            nan,      nan]],
        [[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
```

```

nan, -2.49931443],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -2.81499993],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -2.87813703],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -2.83604563]],
...,
[[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.69957783],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.74166923],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.27866383],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.46807513],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.32075523]],
[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -2.03630903],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.84689773],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.74166923]],
[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],

```

```

[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -0.85774983],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan]]], shape=(9, 24, 114)),
'X_ori': array([[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.25761813],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -1.13134393],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan]],
[[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -2.60454293],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan]],
[[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -2.49931443],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
[      nan,      nan,      nan, ...,      nan,
      nan,      nan],
...,
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,
      nan, -2.81499993],
[-0.63318232, -0.65958335, -0.42120251, ...,      nan,

```

```

        nan, -2.87813703],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -2.83604563]],

...,

[[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -1.69957783],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -1.74166923],
[
        nan,          nan,          nan, ..., nan,
        nan,          nan],

...,

[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -1.27866383],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -1.46807513],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -1.32075523]],

[[
        nan,          nan,          nan, ..., nan,
        nan,          nan],
[
        nan,          nan,          nan, ..., nan,
        nan,          nan],
[
        nan,          nan,          nan, ..., nan,
        nan,          nan],

...,

[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -2.03630903],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -1.84689773],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -1.74166923]],

[[
        nan,          nan,          nan, ..., nan,
        nan,          nan],
[
        nan,          nan,          nan, ..., nan,
        nan,          nan],
[
        nan,          nan,          nan, ..., nan,
        nan,          nan],

...,

[
        nan,          nan,          nan, ..., nan,
        nan,          nan],
[-0.63318232, -0.65958335, -0.42120251, ..., nan,
        nan, -0.85774983],
[
        nan,          nan,          nan, ..., nan,
        nan,          nan]]], shape=(9, 24, 114)))}

```

1.11 Imputación usando SAITS

```
[30]: import torch
from torch.nn import SmoothL1Loss
from torch.optim import Adam
from torch.optim.lr_scheduler import ReduceLROnPlateau
from pypots.imputation import SAITS

saits = SAITS(
    n_steps=24,
    n_features=periodo_1_dataset['n_features'],
    n_layers=3,
    d_model=128,
    d_ffn=256,
    n_heads=4,
    d_k=32,
    d_v=32,
    dropout=0.3,
    attn_dropout=0.2,
    diagonal_attention_mask=True,
    ORT_weight=1,
    MIT_weight=1,

    # Entrenamiento
    batch_size=64,
    epochs=200,
    patience=20,
    num_workers=0,
    device=None,

    saving_path="tutorial_results/imputation/saits",
    model_saving_strategy="best",
)

# Optimizer con menor tasa de aprendizaje y más regularización
optimizer = Adam(saits.model.parameters(), lr=5e-5, weight_decay=1e-4)

# Scheduler que reduce el LR si la pérdida no mejora
scheduler = ReduceLROnPlateau(optimizer, mode="min", factor=0.5, patience=5)

# Función de pérdida SmoothL1Loss en vez de MAE puro
loss_fn = SmoothL1Loss(beta=0.1)
```

```
2025-04-05 17:31:26 [INFO]: No given device, using default device: cpu
2025-04-05 17:31:26 [INFO]: Model files will be saved to
tutorial_results/imputation/saits/20250405_T173126
2025-04-05 17:31:26 [INFO]: Tensorboard file will be saved to
tutorial_results/imputation/saits/20250405_T173126/tensorboard
```

```
2025-04-05 17:31:26 [INFO]: Using customized MAE as the training loss function.
2025-04-05 17:31:26 [INFO]: Using customized MSE as the validation metric
function.
2025-04-05 17:31:26 [INFO]: SAITS initialized with the given hyperparameters,
the number of trainable parameters: 908,800
```

ai4ts v0.0.3 - building AI for unified time-series analysis, <https://time-series.ai>

```
[31]: # Entrenar el modelo
saits.fit(train_set=dataset_for_training, val_set=dataset_for_validating)
```

```
2025-04-05 17:31:26 [INFO]: Epoch 001 - training loss (MAE): 1.5426, validation
MSE: 1.0942
2025-04-05 17:31:26 [INFO]: Epoch 002 - training loss (MAE): 1.4330, validation
MSE: 0.9418
2025-04-05 17:31:26 [INFO]: Epoch 003 - training loss (MAE): 1.3530, validation
MSE: 0.8879
2025-04-05 17:31:26 [INFO]: Epoch 004 - training loss (MAE): 1.2996, validation
MSE: 0.8483
2025-04-05 17:31:26 [INFO]: Epoch 005 - training loss (MAE): 1.2432, validation
MSE: 0.7687
2025-04-05 17:31:26 [INFO]: Epoch 006 - training loss (MAE): 1.1730, validation
MSE: 0.6874
2025-04-05 17:31:26 [INFO]: Epoch 007 - training loss (MAE): 1.1078, validation
MSE: 0.6027
2025-04-05 17:31:26 [INFO]: Epoch 008 - training loss (MAE): 1.0498, validation
```

MSE: 0.5393
2025-04-05 17:31:26 [INFO]: Epoch 009 - training loss (MAE): 1.0252, validation
MSE: 0.5181
2025-04-05 17:31:26 [INFO]: Epoch 010 - training loss (MAE): 1.0017, validation
MSE: 0.5053
2025-04-05 17:31:26 [INFO]: Epoch 011 - training loss (MAE): 0.9837, validation
MSE: 0.4858
2025-04-05 17:31:27 [INFO]: Epoch 012 - training loss (MAE): 0.9740, validation
MSE: 0.4734
2025-04-05 17:31:27 [INFO]: Epoch 013 - training loss (MAE): 0.9555, validation
MSE: 0.4734
2025-04-05 17:31:27 [INFO]: Epoch 014 - training loss (MAE): 0.9406, validation
MSE: 0.4783
2025-04-05 17:31:27 [INFO]: Epoch 015 - training loss (MAE): 0.9365, validation
MSE: 0.4820
2025-04-05 17:31:27 [INFO]: Epoch 016 - training loss (MAE): 0.9114, validation
MSE: 0.4747
2025-04-05 17:31:27 [INFO]: Epoch 017 - training loss (MAE): 0.9090, validation
MSE: 0.4661
2025-04-05 17:31:27 [INFO]: Epoch 018 - training loss (MAE): 0.8966, validation
MSE: 0.4533
2025-04-05 17:31:27 [INFO]: Epoch 019 - training loss (MAE): 0.8957, validation
MSE: 0.4374
2025-04-05 17:31:27 [INFO]: Epoch 020 - training loss (MAE): 0.8930, validation
MSE: 0.4333
2025-04-05 17:31:27 [INFO]: Epoch 021 - training loss (MAE): 0.8783, validation
MSE: 0.4306
2025-04-05 17:31:27 [INFO]: Epoch 022 - training loss (MAE): 0.8723, validation
MSE: 0.4149
2025-04-05 17:31:27 [INFO]: Epoch 023 - training loss (MAE): 0.8726, validation
MSE: 0.4022
2025-04-05 17:31:27 [INFO]: Epoch 024 - training loss (MAE): 0.8632, validation
MSE: 0.4001
2025-04-05 17:31:27 [INFO]: Epoch 025 - training loss (MAE): 0.8640, validation
MSE: 0.4075
2025-04-05 17:31:27 [INFO]: Epoch 026 - training loss (MAE): 0.8488, validation
MSE: 0.4164
2025-04-05 17:31:27 [INFO]: Epoch 027 - training loss (MAE): 0.8520, validation
MSE: 0.3997
2025-04-05 17:31:28 [INFO]: Epoch 028 - training loss (MAE): 0.8442, validation
MSE: 0.3939
2025-04-05 17:31:28 [INFO]: Epoch 029 - training loss (MAE): 0.8427, validation
MSE: 0.3838
2025-04-05 17:31:28 [INFO]: Epoch 030 - training loss (MAE): 0.8444, validation
MSE: 0.3839
2025-04-05 17:31:28 [INFO]: Epoch 031 - training loss (MAE): 0.8298, validation
MSE: 0.4063
2025-04-05 17:31:28 [INFO]: Epoch 032 - training loss (MAE): 0.8389, validation

MSE: 0.3803
 2025-04-05 17:31:28 [INFO]: Epoch 033 - training loss (MAE): 0.8347, validation
 MSE: 0.3956
 2025-04-05 17:31:28 [INFO]: Epoch 034 - training loss (MAE): 0.8297, validation
 MSE: 0.4246
 2025-04-05 17:31:28 [INFO]: Epoch 035 - training loss (MAE): 0.8192, validation
 MSE: 0.4216
 2025-04-05 17:31:28 [INFO]: Epoch 036 - training loss (MAE): 0.8187, validation
 MSE: 0.4240
 2025-04-05 17:31:28 [INFO]: Epoch 037 - training loss (MAE): 0.8213, validation
 MSE: 0.4291
 2025-04-05 17:31:28 [INFO]: Epoch 038 - training loss (MAE): 0.8090, validation
 MSE: 0.4312
 2025-04-05 17:31:28 [INFO]: Epoch 039 - training loss (MAE): 0.8072, validation
 MSE: 0.4362
 2025-04-05 17:31:28 [INFO]: Epoch 040 - training loss (MAE): 0.7978, validation
 MSE: 0.4430
 2025-04-05 17:31:28 [INFO]: Epoch 041 - training loss (MAE): 0.8039, validation
 MSE: 0.4531
 2025-04-05 17:31:28 [INFO]: Epoch 042 - training loss (MAE): 0.7933, validation
 MSE: 0.4615
 2025-04-05 17:31:28 [INFO]: Epoch 043 - training loss (MAE): 0.7932, validation
 MSE: 0.4570
 2025-04-05 17:31:29 [INFO]: Epoch 044 - training loss (MAE): 0.7928, validation
 MSE: 0.4405
 2025-04-05 17:31:29 [INFO]: Epoch 045 - training loss (MAE): 0.7872, validation
 MSE: 0.4373
 2025-04-05 17:31:29 [INFO]: Epoch 046 - training loss (MAE): 0.7843, validation
 MSE: 0.4352
 2025-04-05 17:31:29 [INFO]: Epoch 047 - training loss (MAE): 0.7774, validation
 MSE: 0.4313
 2025-04-05 17:31:29 [INFO]: Epoch 048 - training loss (MAE): 0.7776, validation
 MSE: 0.4450
 2025-04-05 17:31:29 [INFO]: Epoch 049 - training loss (MAE): 0.7667, validation
 MSE: 0.4597
 2025-04-05 17:31:29 [INFO]: Epoch 050 - training loss (MAE): 0.7724, validation
 MSE: 0.4572
 2025-04-05 17:31:29 [INFO]: Epoch 051 - training loss (MAE): 0.7693, validation
 MSE: 0.4637
 2025-04-05 17:31:29 [INFO]: Epoch 052 - training loss (MAE): 0.7654, validation
 MSE: 0.4531
 2025-04-05 17:31:29 [INFO]: Exceeded the training patience. Terminating the
 training procedure...
 2025-04-05 17:31:29 [INFO]: Finished training. The best model is from epoch#32.
 2025-04-05 17:31:29 [INFO]: Saved the model to
 tutorial_results/imputation/saits/20250405_T173126/SAITS.pypots


```
[32]: saits_results = saits.predict(dataset_for_testing)
      saits_imputation = saits_results["imputation"]
```

```
[33]: saits_imputation
```

```
[33]: array([[[-0.6331823 , -0.65958333, -0.4212025 , ...,  0.21799918,
               -0.86489993, -0.75252134],
              [-0.48855716, -0.5043099 , -0.22803478, ...,  0.23191401,
               -0.71644557,  0.68805885],
              [-0.4738921 , -0.4600316 , -0.20835698, ...,  0.24076073,
               -0.7308186 ,  0.7103846 ],
              ...,
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.14702162,
               -0.76143336, -1.0050697 ],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.16843504,
               -0.77412117, -0.89984125],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.1729429 ,
               -0.7738409 , -0.89984125]],

            [[[-0.6331823 , -0.65958333, -0.4212025 , ...,  0.23964238,
               -0.8286025 , -0.89984125],
              [-0.49002057, -0.511963 , -0.18065356, ...,  0.27190527,
               -0.68703943,  0.7546922 ],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.23585851,
               -0.8457831 , -0.8577498 ],
              ...,
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.14836848,
               -0.74180174, -0.75252134],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.1754126 ,
               -0.7763852 , -0.64729285],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.1816256 ,
               -0.7849089 , -0.66833854]],

            [[[-0.6331823 , -0.65958333, -0.4212025 , ...,  0.24902041,
               -0.76798004, -0.75252134],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.25121176,
               -0.7707398 , -0.81565845],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.25400108,
               -0.7756682 , -0.81565845],
              ...,
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.18910095,
               -0.69420785, -1.5733036 ],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.19751994,
               -0.7013957 , -1.6995778 ],
              [-0.6331823 , -0.65958333, -0.4212025 , ...,  0.20458896,
               -0.7183108 , -1.7416692 ]],
```

```

...,
[[-0.6331823 , -0.65958333, -0.4212025 , ..., 0.3002844 ,
  -0.81949073, 0.13139807],
 [-0.6331823 , -0.65958333, -0.4212025 , ..., 0.29616287,
  -0.8193784 , 0.13139807],
 [-0.6331823 , -0.65958333, -0.4212025 , ..., 0.30980903,
  -0.82339036, 0.02616957],
 ...,
 [-0.6331823 , -0.65958333, -0.4212025 , ..., 0.23378396,
  -0.7502065 , 0.06826097],
 [-0.6331823 , -0.65958333, -0.4212025 , ..., 0.2545137 ,
  -0.7682805 , -0.03696753],
 [-0.6331823 , -0.65958333, -0.4212025 , ..., 0.25863338,
  -0.7731025 , 0.00512387]],

[[-0.6331823 , -0.65958333, -0.4212025 , ..., 0.18534805,
  -0.73941195, -0.14219603],
 [-0.6331823 , -0.65958333, -0.4212025 , ..., 0.180597 ,
  -0.7420231 , -0.14219603],
 [-0.33638886, -0.372639 , -0.32736242, ..., -0.01380051,
  -0.23567343, 0.29755723],
 ...,
 [-0.4396659 , -0.5160811 , -0.2977668 , ..., -0.11853661,
  -0.24018617, 0.21875867],
 [-0.45788404, -0.5346749 , -0.3195543 , ..., -0.06202656,
  -0.2901658 , 0.24406305],
 [-0.4511907 , -0.5483401 , -0.34234002, ..., -0.05495573,
  -0.28955105, 0.25314915]],

[[-0.48971075, -0.53938496, -0.3727227 , ..., -0.05690437,
  -0.3676453 , 0.09937427],
 [-0.44847813, -0.4796047 , -0.35917658, ..., -0.07508937,
  -0.31244358, 0.09614938],
 [-0.42090783, -0.40542156, -0.35240746, ..., -0.11665925,
  -0.27531227, 0.06980484],
 ...,
 [-0.49692684, -0.53738993, -0.3309431 , ..., -0.22288331,
  -0.25449964, 0.02980927],
 [-0.5184211 , -0.55914694, -0.3523273 , ..., -0.16613135,
  -0.30580822, 0.05758762],
 [-0.5118096 , -0.5708609 , -0.37612867, ..., -0.16043808,
  -0.30702323, 0.06267622]]], shape=(12, 24, 114), dtype=float32)

```

```

[34]: # Crear el índice con las fechas y horas de 2023-03-03 00:00:00 hasta
      ↪ 2023-03-14 23:00:00

```

```
date_range = pd.date_range(start="2023-03-03 00:00:00", end="2023-03-14 23:00:00", freq="h", tz="UTC")
```

```
[35]: import pandas as pd
import numpy as np

# Supongamos que 'saits_imputation' es tu numpy array de shape (12, 24, 114)
# Reestructuramos el array para convertirlo en un formato adecuado para sicies
↳ temporales multivariantes
# Transponemos y aplanamos para que tengamos (12 * 24, 114)

X_resaped_saits = saits_imputation.reshape(-1, len(features)) # Ahora
↳ X_resaped tiene la forma (12 * 24, 114)

# Crear el índice con las fechas y horas de 2023-03-03 00:00:00 hasta
↳ 2023-03-14 23:00:00
date_range = pd.date_range(start="2023-03-03 00:00:00", end="2023-03-14 23:00:00", freq="h", tz="UTC")

# Creamos el DataFrame
periodo_1_imputed = pd.DataFrame(X_resaped_saits, columns=features,
↳ index=date_range)

# Mostrar las primeras filas del DataFrame
periodo_1_imputed
```

```
[35]: vehicles_PAM_1_OUT \
2023-03-03 00:00:00+00:00 -0.633182
2023-03-03 01:00:00+00:00 -0.488557
2023-03-03 02:00:00+00:00 -0.473892
2023-03-03 03:00:00+00:00 -0.508814
2023-03-03 04:00:00+00:00 -0.580272
...
2023-03-14 19:00:00+00:00 -0.358429
2023-03-14 20:00:00+00:00 -0.448067
2023-03-14 21:00:00+00:00 -0.496927
2023-03-14 22:00:00+00:00 -0.518421
2023-03-14 23:00:00+00:00 -0.511810

vehicles_PAM_1_OUT_Zona_Granada \
2023-03-03 00:00:00+00:00 -0.659583
2023-03-03 01:00:00+00:00 -0.504310
2023-03-03 02:00:00+00:00 -0.460032
2023-03-03 03:00:00+00:00 -0.471504
2023-03-03 04:00:00+00:00 -0.511620
```

...	...
2023-03-14 19:00:00+00:00	-0.388518
2023-03-14 20:00:00+00:00	-0.496701
2023-03-14 21:00:00+00:00	-0.537390
2023-03-14 22:00:00+00:00	-0.559147
2023-03-14 23:00:00+00:00	-0.570861

	vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
2023-03-03 00:00:00+00:00	-0.421203
2023-03-03 01:00:00+00:00	-0.228035
2023-03-03 02:00:00+00:00	-0.208357
2023-03-03 03:00:00+00:00	-0.207817
2023-03-03 04:00:00+00:00	-0.212013

...	...
2023-03-14 19:00:00+00:00	-0.374627
2023-03-14 20:00:00+00:00	-0.331658
2023-03-14 21:00:00+00:00	-0.330943
2023-03-14 22:00:00+00:00	-0.352327
2023-03-14 23:00:00+00:00	-0.376129

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR \
2023-03-03 00:00:00+00:00	-0.448159
2023-03-03 01:00:00+00:00	-0.655588
2023-03-03 02:00:00+00:00	-0.672568
2023-03-03 03:00:00+00:00	-0.668624
2023-03-03 04:00:00+00:00	-0.646504

...	...
2023-03-14 19:00:00+00:00	-0.320651
2023-03-14 20:00:00+00:00	-0.341544
2023-03-14 21:00:00+00:00	-0.370450
2023-03-14 22:00:00+00:00	-0.403404
2023-03-14 23:00:00+00:00	-0.425676

	vehicles_PAM_1_OUT_Extranjero \
2023-03-03 00:00:00+00:00	-0.526281
2023-03-03 01:00:00+00:00	-0.421564
2023-03-03 02:00:00+00:00	-0.448306
2023-03-03 03:00:00+00:00	-0.522352
2023-03-03 04:00:00+00:00	-0.582507

...	...
2023-03-14 19:00:00+00:00	-0.280527
2023-03-14 20:00:00+00:00	-0.362790
2023-03-14 21:00:00+00:00	-0.396456
2023-03-14 22:00:00+00:00	-0.411403
2023-03-14 23:00:00+00:00	-0.410993

vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \

2023-03-03 00:00:00+00:00	-0.428631
2023-03-03 01:00:00+00:00	-0.277981
2023-03-03 02:00:00+00:00	-0.255001
2023-03-03 03:00:00+00:00	-0.280407
2023-03-03 04:00:00+00:00	-0.331936
...	...
2023-03-14 19:00:00+00:00	-0.269250
2023-03-14 20:00:00+00:00	-0.238529
2023-03-14 21:00:00+00:00	-0.222409
2023-03-14 22:00:00+00:00	-0.220935
2023-03-14 23:00:00+00:00	-0.222955

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
2023-03-03 00:00:00+00:00	-0.436018
2023-03-03 01:00:00+00:00	-0.426228
2023-03-03 02:00:00+00:00	-0.425494
2023-03-03 03:00:00+00:00	-0.419495
2023-03-03 04:00:00+00:00	-0.406764
...	...
2023-03-14 19:00:00+00:00	-0.293994
2023-03-14 20:00:00+00:00	-0.311557
2023-03-14 21:00:00+00:00	-0.337649
2023-03-14 22:00:00+00:00	-0.370596
2023-03-14 23:00:00+00:00	-0.393785

	vehicles_PAM_1_OUT_Zona_Otras \
2023-03-03 00:00:00+00:00	-0.094664
2023-03-03 01:00:00+00:00	-0.133020
2023-03-03 02:00:00+00:00	-0.097417
2023-03-03 03:00:00+00:00	-0.066859
2023-03-03 04:00:00+00:00	-0.066334
...	...
2023-03-14 19:00:00+00:00	-0.085900
2023-03-14 20:00:00+00:00	-0.080902
2023-03-14 21:00:00+00:00	-0.076679
2023-03-14 22:00:00+00:00	-0.063009
2023-03-14 23:00:00+00:00	-0.061661

	vehicles_PAM_1_OUT_5_seats \
2023-03-03 00:00:00+00:00	-0.611788
2023-03-03 01:00:00+00:00	-0.514625
2023-03-03 02:00:00+00:00	-0.424393
2023-03-03 03:00:00+00:00	-0.377866
2023-03-03 04:00:00+00:00	-0.399935
...	...
2023-03-14 19:00:00+00:00	-0.334248
2023-03-14 20:00:00+00:00	-0.446869

2023-03-14 21:00:00+00:00	-0.462296
2023-03-14 22:00:00+00:00	-0.434302
2023-03-14 23:00:00+00:00	-0.398097

	vehicles_PAM_1_OUT_+5_seats	...	NO	\
2023-03-03 00:00:00+00:00	-0.447362	...	-0.790552	
2023-03-03 01:00:00+00:00	-0.370036	...	-0.676735	
2023-03-03 02:00:00+00:00	-0.376942	...	-0.716772	
2023-03-03 03:00:00+00:00	-0.428906	...	-0.726833	
2023-03-03 04:00:00+00:00	-0.492656	...	-0.687898	
...	...	...	...	
2023-03-14 19:00:00+00:00	-0.426776	...	-0.244397	
2023-03-14 20:00:00+00:00	-0.439781	...	-0.345095	
2023-03-14 21:00:00+00:00	-0.458835	...	-0.412895	
2023-03-14 22:00:00+00:00	-0.471877	...	-0.460906	
2023-03-14 23:00:00+00:00	-0.472977	...	-0.483466	

	O3	PM10	eBC_ff	eBC_bb	TEMP	\
2023-03-03 00:00:00+00:00	0.214906	-0.606039	-0.326645	-0.216618	-1.325130	
2023-03-03 01:00:00+00:00	0.015443	-0.037224	-0.682753	-0.240743	-0.160658	
2023-03-03 02:00:00+00:00	0.089110	-0.023927	-0.647735	-0.290139	-0.197552	
2023-03-03 03:00:00+00:00	0.074709	-0.073719	-0.647475	-0.327573	-0.272047	
2023-03-03 04:00:00+00:00	-0.010049	-0.156217	-0.678060	-0.351217	-0.350391	
...	...	...	...	...	...	
2023-03-14 19:00:00+00:00	-0.216631	-0.420333	-0.265622	-0.160879	-0.400020	
2023-03-14 20:00:00+00:00	-0.282154	-0.403607	-0.400085	-0.197294	-0.461997	
2023-03-14 21:00:00+00:00	-0.300137	-0.367676	-0.475338	-0.220546	-0.491456	
2023-03-14 22:00:00+00:00	-0.279220	-0.352899	-0.514589	-0.226912	-0.507959	
2023-03-14 23:00:00+00:00	-0.250485	-0.360275	-0.521645	-0.216357	-0.503287	

	RH	WS	WD	PRES
2023-03-03 00:00:00+00:00	-0.182831	0.217999	-0.864900	-0.752521
2023-03-03 01:00:00+00:00	-0.334508	0.231914	-0.716446	0.688059
2023-03-03 02:00:00+00:00	-0.320854	0.240761	-0.730819	0.710385
2023-03-03 03:00:00+00:00	-0.293899	0.246058	-0.786408	0.673074
2023-03-03 04:00:00+00:00	-0.274882	0.229028	-0.865303	0.599800
...	...	...	...	...
2023-03-14 19:00:00+00:00	0.610849	-0.369736	-0.069293	-0.101972
2023-03-14 20:00:00+00:00	0.572000	-0.292151	-0.183442	-0.025350
2023-03-14 21:00:00+00:00	0.496609	-0.222883	-0.254500	0.029809
2023-03-14 22:00:00+00:00	0.446708	-0.166131	-0.305808	0.057588
2023-03-14 23:00:00+00:00	0.443654	-0.160438	-0.307023	0.062676

[288 rows x 114 columns]

```
[36]: import pandas as pd
import numpy as np
```

```

# Supongamos que 'saits_imputation' es tu numpy array de shape (12, 24, 114)
# Reestructuramos el array para convertirlo en un formato adecuado para series
↳ temporales multivariantes
# Transponemos y aplanamos para que tengamos (12 * 24, 114)

X_resaped = X_imputed.reshape(-1, len(features)) # Ahora X_resaped tiene la
↳ forma (12 * 24, 114)

# Crear el índice con las fechas y horas de 2023-03-03 00:00:00 hasta
↳ 2023-03-14 23:00:00
date_range = pd.date_range(start="2023-03-03 00:00:00", end="2023-03-14 23:00:
↳ 00", freq="h", tz="UTC")

# Creamos el DataFrame
periodo_1_median_imputed = pd.DataFrame(X_resaped, columns=features,
↳ index=date_range)

# Mostrar las primeras filas del DataFrame
periodo_1_median_imputed

```

[36]:

	vehicles_PAM_1_OUT \
2023-03-03 00:00:00+00:00	-0.633182
2023-03-03 01:00:00+00:00	-0.633182
2023-03-03 02:00:00+00:00	-0.633182
2023-03-03 03:00:00+00:00	-0.633182
2023-03-03 04:00:00+00:00	-0.633182
...	...
2023-03-14 19:00:00+00:00	-0.633182
2023-03-14 20:00:00+00:00	-0.633182
2023-03-14 21:00:00+00:00	-0.633182
2023-03-14 22:00:00+00:00	-0.633182
2023-03-14 23:00:00+00:00	-0.633182

	vehicles_PAM_1_OUT_Zona_Granada \
2023-03-03 00:00:00+00:00	-0.659583
2023-03-03 01:00:00+00:00	-0.659583
2023-03-03 02:00:00+00:00	-0.659583
2023-03-03 03:00:00+00:00	-0.659583
2023-03-03 04:00:00+00:00	-0.659583
...	...
2023-03-14 19:00:00+00:00	-0.659583
2023-03-14 20:00:00+00:00	-0.659583
2023-03-14 21:00:00+00:00	-0.659583
2023-03-14 22:00:00+00:00	-0.659583
2023-03-14 23:00:00+00:00	-0.659583

	vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
2023-03-03 00:00:00+00:00	-0.421203
2023-03-03 01:00:00+00:00	-0.421203
2023-03-03 02:00:00+00:00	-0.421203
2023-03-03 03:00:00+00:00	-0.421203
2023-03-03 04:00:00+00:00	-0.421203
...	...
2023-03-14 19:00:00+00:00	-0.421203
2023-03-14 20:00:00+00:00	-0.421203
2023-03-14 21:00:00+00:00	-0.421203
2023-03-14 22:00:00+00:00	-0.421203
2023-03-14 23:00:00+00:00	-0.421203

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR \
2023-03-03 00:00:00+00:00	-0.448159
2023-03-03 01:00:00+00:00	-0.448159
2023-03-03 02:00:00+00:00	-0.448159
2023-03-03 03:00:00+00:00	-0.448159
2023-03-03 04:00:00+00:00	-0.448159
...	...
2023-03-14 19:00:00+00:00	-0.448159
2023-03-14 20:00:00+00:00	-0.448159
2023-03-14 21:00:00+00:00	-0.448159
2023-03-14 22:00:00+00:00	-0.448159
2023-03-14 23:00:00+00:00	-0.448159

	vehicles_PAM_1_OUT_Extranjero \
2023-03-03 00:00:00+00:00	-0.526281
2023-03-03 01:00:00+00:00	-0.526281
2023-03-03 02:00:00+00:00	-0.526281
2023-03-03 03:00:00+00:00	-0.526281
2023-03-03 04:00:00+00:00	-0.526281
...	...
2023-03-14 19:00:00+00:00	-0.526281
2023-03-14 20:00:00+00:00	-0.526281
2023-03-14 21:00:00+00:00	-0.526281
2023-03-14 22:00:00+00:00	-0.526281
2023-03-14 23:00:00+00:00	-0.526281

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
2023-03-03 00:00:00+00:00	-0.428631
2023-03-03 01:00:00+00:00	-0.428631
2023-03-03 02:00:00+00:00	-0.428631
2023-03-03 03:00:00+00:00	-0.428631
2023-03-03 04:00:00+00:00	-0.428631
...	...
2023-03-14 19:00:00+00:00	-0.428631

2023-03-14 20:00:00+00:00	-0.428631
2023-03-14 21:00:00+00:00	-0.428631
2023-03-14 22:00:00+00:00	-0.428631
2023-03-14 23:00:00+00:00	-0.428631

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
2023-03-03 00:00:00+00:00	-0.436018
2023-03-03 01:00:00+00:00	-0.436018
2023-03-03 02:00:00+00:00	-0.436018
2023-03-03 03:00:00+00:00	-0.436018
2023-03-03 04:00:00+00:00	-0.436018
...	...
2023-03-14 19:00:00+00:00	-0.436018
2023-03-14 20:00:00+00:00	-0.436018
2023-03-14 21:00:00+00:00	-0.436018
2023-03-14 22:00:00+00:00	-0.436018
2023-03-14 23:00:00+00:00	-0.436018

	vehicles_PAM_1_OUT_Zona_Otras \
2023-03-03 00:00:00+00:00	-0.094664
2023-03-03 01:00:00+00:00	-0.094664
2023-03-03 02:00:00+00:00	-0.094664
2023-03-03 03:00:00+00:00	-0.094664
2023-03-03 04:00:00+00:00	-0.094664
...	...
2023-03-14 19:00:00+00:00	-0.094664
2023-03-14 20:00:00+00:00	-0.094664
2023-03-14 21:00:00+00:00	-0.094664
2023-03-14 22:00:00+00:00	-0.094664
2023-03-14 23:00:00+00:00	-0.094664

	vehicles_PAM_1_OUT_5_seats \
2023-03-03 00:00:00+00:00	-0.611788
2023-03-03 01:00:00+00:00	-0.611788
2023-03-03 02:00:00+00:00	-0.611788
2023-03-03 03:00:00+00:00	-0.611788
2023-03-03 04:00:00+00:00	-0.611788
...	...
2023-03-14 19:00:00+00:00	-0.611788
2023-03-14 20:00:00+00:00	-0.611788
2023-03-14 21:00:00+00:00	-0.611788
2023-03-14 22:00:00+00:00	-0.611788
2023-03-14 23:00:00+00:00	-0.611788

	vehicles_PAM_1_OUT_+5_seats ...	NO \
2023-03-03 00:00:00+00:00	-0.447362	... -0.790552
2023-03-03 01:00:00+00:00	-0.447362	... -0.639947

2023-03-03 02:00:00+00:00	-0.447362	...	-0.790552
2023-03-03 03:00:00+00:00	-0.447362	...	-0.790552
2023-03-03 04:00:00+00:00	-0.447362	...	-0.790552
...	...	...	...
2023-03-14 19:00:00+00:00	-0.447362	...	-0.112829
2023-03-14 20:00:00+00:00	-0.447362	...	-0.414040
2023-03-14 21:00:00+00:00	-0.447362	...	-0.489342
2023-03-14 22:00:00+00:00	-0.447362	...	-0.639947
2023-03-14 23:00:00+00:00	-0.447362	...	-0.639947

	O3	PM10	eBC_ff	eBC_bb	TEMP \
2023-03-03 00:00:00+00:00	0.214906	-0.606039	-0.326645	-0.216618	-1.325130
2023-03-03 01:00:00+00:00	-0.845414	-0.758898	-0.326645	-1.076590	0.934599
2023-03-03 02:00:00+00:00	0.613658	-0.549926	-0.788132	-0.216618	-0.162357
2023-03-03 03:00:00+00:00	-0.990415	-0.389327	-0.788132	-1.076590	0.846843
2023-03-03 04:00:00+00:00	0.260219	-0.857580	-0.788132	-0.216618	-0.140417
...	...	...	...	...	...
2023-03-14 19:00:00+00:00	0.020061	-0.191964	1.519305	-0.216618	1.033325
2023-03-14 20:00:00+00:00	0.097092	-0.565405	-0.326645	-0.216618	0.583573
2023-03-14 21:00:00+00:00	0.024592	-0.710525	-0.788132	-0.216618	0.759086
2023-03-14 22:00:00+00:00	0.078967	-0.629258	0.134843	0.213368	0.309334
2023-03-14 23:00:00+00:00	-0.455724	-0.404806	-0.788132	-0.216618	0.759086

	RH	WS	WD	PRES
2023-03-03 00:00:00+00:00	-0.182831	0.048025	0.048025	-0.752521
2023-03-03 01:00:00+00:00	-0.431523	0.048025	0.048025	-0.142196
2023-03-03 02:00:00+00:00	-0.611825	0.048025	0.048025	-0.815658
2023-03-03 03:00:00+00:00	-0.686432	0.048025	0.048025	0.320809
2023-03-03 04:00:00+00:00	-0.245004	0.048025	0.048025	-0.899841
...	...	...	...	...
2023-03-14 19:00:00+00:00	0.550810	0.048025	0.048025	-1.036638
2023-03-14 20:00:00+00:00	0.656504	0.048025	0.048025	-0.962978
2023-03-14 21:00:00+00:00	-0.151745	0.048025	0.048025	-0.752521
2023-03-14 22:00:00+00:00	0.224402	0.048025	0.048025	-0.752521
2023-03-14 23:00:00+00:00	-0.089572	0.048025	0.048025	-0.668339

[288 rows x 114 columns]

```
[37]: import pandas as pd
import numpy as np

# Supongamos que 'saits_imputation' es tu numpy array de shape (12, 24, 114)
# Reestructuramos el array para convertirlo en un formato adecuado para series
↳ temporales multivariantes
# Transponemos y aplanamos para que tengamos (12 * 24, 114)
```

```

X_resaped_mean = X_imputed_mean.reshape(-1, len(features)) # Ahora X_resaped_
↳ tiene la forma (12 * 24, 114)

# Crear el índice con las fechas y horas de 2023-03-03 00:00:00 hasta
↳ 2023-03-14 23:00:00
date_range = pd.date_range(start="2023-03-03 00:00:00", end="2023-03-14 23:00:
↳ 00", freq="h", tz="UTC")

# Creamos el DataFrame
periodo_1_mean_imputed = pd.DataFrame(X_resaped_mean, columns=features,
↳ index=date_range)

# Mostrar las primeras filas del DataFrame
periodo_1_mean_imputed

```

```

[37]:
vehicles_PAM_1_OUT \
2023-03-03 00:00:00+00:00      -0.633182
2023-03-03 01:00:00+00:00      -0.633182
2023-03-03 02:00:00+00:00      -0.633182
2023-03-03 03:00:00+00:00      -0.633182
2023-03-03 04:00:00+00:00      -0.633182
...
2023-03-14 19:00:00+00:00      -0.491266
2023-03-14 20:00:00+00:00      -0.623045
2023-03-14 21:00:00+00:00      -0.633182
2023-03-14 22:00:00+00:00      -0.633182
2023-03-14 23:00:00+00:00      -0.633182

vehicles_PAM_1_OUT_Zona_Granada \
2023-03-03 00:00:00+00:00      -0.659583
2023-03-03 01:00:00+00:00      -0.659583
2023-03-03 02:00:00+00:00      -0.659583
2023-03-03 03:00:00+00:00      -0.659583
2023-03-03 04:00:00+00:00      -0.659583
...
2023-03-14 19:00:00+00:00      -0.489966
2023-03-14 20:00:00+00:00      -0.659583
2023-03-14 21:00:00+00:00      -0.659583
2023-03-14 22:00:00+00:00      -0.659583
2023-03-14 23:00:00+00:00      -0.659583

vehicles_PAM_1_OUT_Zona_Catalunia_y_Otras \
2023-03-03 00:00:00+00:00      -0.421203
2023-03-03 01:00:00+00:00      -0.421203
2023-03-03 02:00:00+00:00      -0.421203
2023-03-03 03:00:00+00:00      -0.421203
2023-03-03 04:00:00+00:00      -0.421203

```

...	...
2023-03-14 19:00:00+00:00	-0.186478
2023-03-14 20:00:00+00:00	-0.303840
2023-03-14 21:00:00+00:00	-0.421203
2023-03-14 22:00:00+00:00	-0.421203
2023-03-14 23:00:00+00:00	-0.421203

	vehicles_PAM_1_OUT_Zona_Andalucia_no_GR \
2023-03-03 00:00:00+00:00	-0.448159
2023-03-03 01:00:00+00:00	-0.448159
2023-03-03 02:00:00+00:00	-0.448159
2023-03-03 03:00:00+00:00	-0.448159
2023-03-03 04:00:00+00:00	-0.448159

...	...
2023-03-14 19:00:00+00:00	-0.401987
2023-03-14 20:00:00+00:00	-0.448159
2023-03-14 21:00:00+00:00	-0.448159
2023-03-14 22:00:00+00:00	-0.448159
2023-03-14 23:00:00+00:00	-0.448159

	vehicles_PAM_1_OUT_Extranjero \
2023-03-03 00:00:00+00:00	-0.526281
2023-03-03 01:00:00+00:00	-0.526281
2023-03-03 02:00:00+00:00	-0.526281
2023-03-03 03:00:00+00:00	-0.526281
2023-03-03 04:00:00+00:00	-0.526281

...	...
2023-03-14 19:00:00+00:00	-0.471694
2023-03-14 20:00:00+00:00	-0.526281
2023-03-14 21:00:00+00:00	-0.526281
2023-03-14 22:00:00+00:00	-0.526281
2023-03-14 23:00:00+00:00	-0.526281

	vehicles_PAM_1_OUT_Zona_Comunidad_de_Madrid \
2023-03-03 00:00:00+00:00	-0.428631
2023-03-03 01:00:00+00:00	-0.428631
2023-03-03 02:00:00+00:00	-0.428631
2023-03-03 03:00:00+00:00	-0.428631
2023-03-03 04:00:00+00:00	-0.428631

...	...
2023-03-14 19:00:00+00:00	-0.328726
2023-03-14 20:00:00+00:00	-0.428631
2023-03-14 21:00:00+00:00	-0.428631
2023-03-14 22:00:00+00:00	-0.428631
2023-03-14 23:00:00+00:00	-0.428631

	vehicles_PAM_1_OUT_Zona_Extremadura_y_Otras \
--	---

2023-03-03 00:00:00+00:00	-0.436018
2023-03-03 01:00:00+00:00	-0.436018
2023-03-03 02:00:00+00:00	-0.436018
2023-03-03 03:00:00+00:00	-0.436018
2023-03-03 04:00:00+00:00	-0.436018
...	...
2023-03-14 19:00:00+00:00	-0.327368
2023-03-14 20:00:00+00:00	-0.436018
2023-03-14 21:00:00+00:00	-0.436018
2023-03-14 22:00:00+00:00	-0.436018
2023-03-14 23:00:00+00:00	-0.436018

	vehicles_PAM_1_OUT_Zona_Otras \
2023-03-03 00:00:00+00:00	-0.094664
2023-03-03 01:00:00+00:00	-0.094664
2023-03-03 02:00:00+00:00	-0.094664
2023-03-03 03:00:00+00:00	-0.094664
2023-03-03 04:00:00+00:00	-0.094664
...	...
2023-03-14 19:00:00+00:00	-0.094664
2023-03-14 20:00:00+00:00	-0.094664
2023-03-14 21:00:00+00:00	-0.094664
2023-03-14 22:00:00+00:00	-0.094664
2023-03-14 23:00:00+00:00	-0.094664

	vehicles_PAM_1_OUT_5_seats \
2023-03-03 00:00:00+00:00	-0.611788
2023-03-03 01:00:00+00:00	-0.611788
2023-03-03 02:00:00+00:00	-0.611788
2023-03-03 03:00:00+00:00	-0.611788
2023-03-03 04:00:00+00:00	-0.611788
...	...
2023-03-14 19:00:00+00:00	-0.443552
2023-03-14 20:00:00+00:00	-0.597768
2023-03-14 21:00:00+00:00	-0.611788
2023-03-14 22:00:00+00:00	-0.611788
2023-03-14 23:00:00+00:00	-0.611788

	vehicles_PAM_1_OUT_+5_seats ...	NO \
2023-03-03 00:00:00+00:00	-0.447362 ...	-0.790552
2023-03-03 01:00:00+00:00	-0.447362 ...	-0.670068
2023-03-03 02:00:00+00:00	-0.447362 ...	-0.840754
2023-03-03 03:00:00+00:00	-0.447362 ...	-0.790552
2023-03-03 04:00:00+00:00	-0.447362 ...	-0.840754
...	...	...
2023-03-14 19:00:00+00:00	-0.319216 ...	-0.052587
2023-03-14 20:00:00+00:00	-0.447362 ...	-0.459221

2023-03-14 21:00:00+00:00						-0.447362	...	-0.556278
2023-03-14 22:00:00+00:00						-0.447362	...	-0.665048
2023-03-14 23:00:00+00:00						-0.447362	...	-0.704492
	O3	PM10	eBC_ff	eBC_bb	TEMP	\		
2023-03-03 00:00:00+00:00	0.214906	-0.606039	-0.326645	-0.216618	-1.325130			
2023-03-03 01:00:00+00:00	-0.598912	-0.730842	-0.142050	-0.904596	0.807352			
2023-03-03 02:00:00+00:00	0.553241	-0.549926	-0.634303	-0.216618	0.122852			
2023-03-03 03:00:00+00:00	-0.990415	-0.389327	-0.788132	-1.076590	0.846843			
2023-03-03 04:00:00+00:00	0.100113	-0.849840	-0.634303	-0.503275	0.071661			
...	...	...	...	...	...			
2023-03-14 19:00:00+00:00	-0.129472	-0.227899	1.473157	0.127371	1.009192			
2023-03-14 20:00:00+00:00	0.033654	-0.482203	-0.018986	0.452250	0.703020			
2023-03-14 21:00:00+00:00	0.143412	-0.694493	-0.429198	0.356697	0.812715			
2023-03-14 22:00:00+00:00	-0.248795	-0.556505	0.134843	0.213368	0.525069			
2023-03-14 23:00:00+00:00	-0.270589	-0.355788	-0.326645	-0.339471	0.611781			
	RH	WS	WD	PRES				
2023-03-03 00:00:00+00:00	-0.182831	0.048025	0.048025	-0.752521				
2023-03-03 01:00:00+00:00	-0.099519	0.048025	0.048025	-0.361071				
2023-03-03 02:00:00+00:00	-0.425306	0.048025	0.048025	-0.549080				
2023-03-03 03:00:00+00:00	-0.686432	0.048025	0.048025	0.320809				
2023-03-03 04:00:00+00:00	-0.493696	0.048025	0.048025	-0.612217				
...	...	...	...	...				
2023-03-14 19:00:00+00:00	0.333826	0.048025	0.048025	-0.876691				
2023-03-14 20:00:00+00:00	0.396621	0.048025	0.048025	-0.813554				
2023-03-14 21:00:00+00:00	0.160502	0.048025	0.048025	-0.656646				
2023-03-14 22:00:00+00:00	0.326987	0.048025	0.048025	-0.654308				
2023-03-14 23:00:00+00:00	0.007240	0.048025	0.048025	-0.508993				

[288 rows x 114 columns]

1.12 Backwards y Forward Fill

```
[38]: import pandas as pd
import numpy as np

X_imputed_ori = dataset_for_testing['X'].copy()

X_resshaped_fill = X_imputed_ori.reshape(-1, len(features)) # Ahora X_resshaped_
    ↪ tiene la forma (12 * 24, 114)

# Crear el índice con las fechas y horas de 2023-03-03 00:00:00 hasta_
    ↪ 2023-03-14 23:00:00
date_range = pd.date_range(start="2023-03-03 00:00:00", end="2023-03-14 23:00:
    ↪ 00", freq="h", tz="UTC")
```

```

# Creamos el DataFrame
periodo_1_fill = pd.DataFrame(X_resaped_fill, columns=features,
    ↪index=date_range)

X_imputed_backwards = periodo_1_fill.bfill(axis=0)
X_imputed_backwards = X_imputed_backwards.ffill(axis=0)

X_imputed_forward = periodo_1_fill.ffill(axis=0)
X_imputed_forward = X_imputed_forward.bfill(axis=0)

# Eliminar las columnas 'WS' y 'WD'
X_imputed_backwards_clean = X_imputed_backwards.drop(columns=["WS", "WD"])
X_imputed_forward_clean = X_imputed_forward.drop(columns=["WS", "WD"])

pd.reset_option('display.max_columns')
pd.reset_option('display.max_rows')

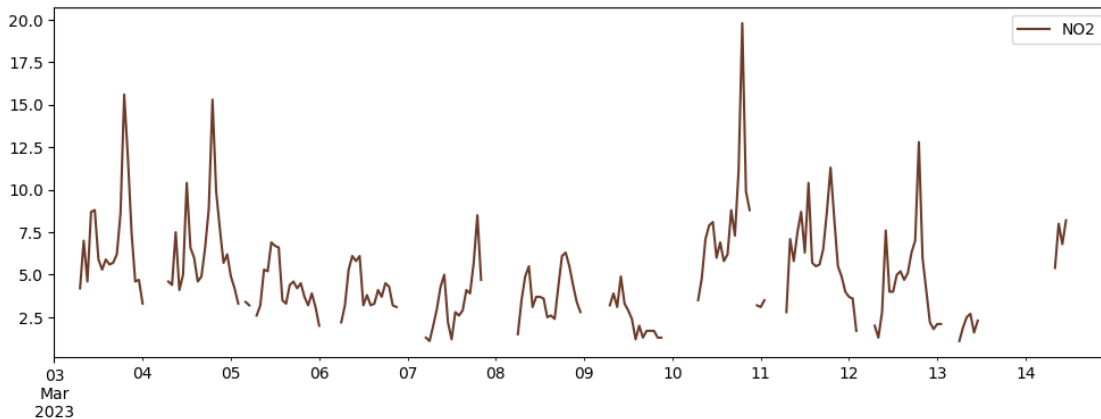
```

1.13 Comparación de gráficas con los datos imputados

1.14 Serie Temporal NO2 con Datos Faltantes

```
[39]: periodo_1_df.loc["2023-03-03":"2023-03-14"][["NO2"]].plot(color='#6c3b2a')
```

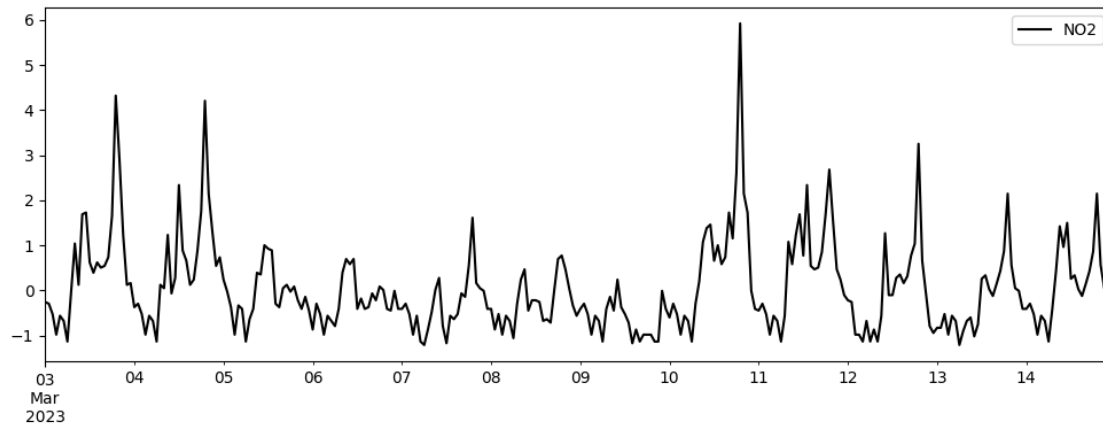
[39]: <Axes: >



1.15 Serie Temporal NO2 imputada con la mediana

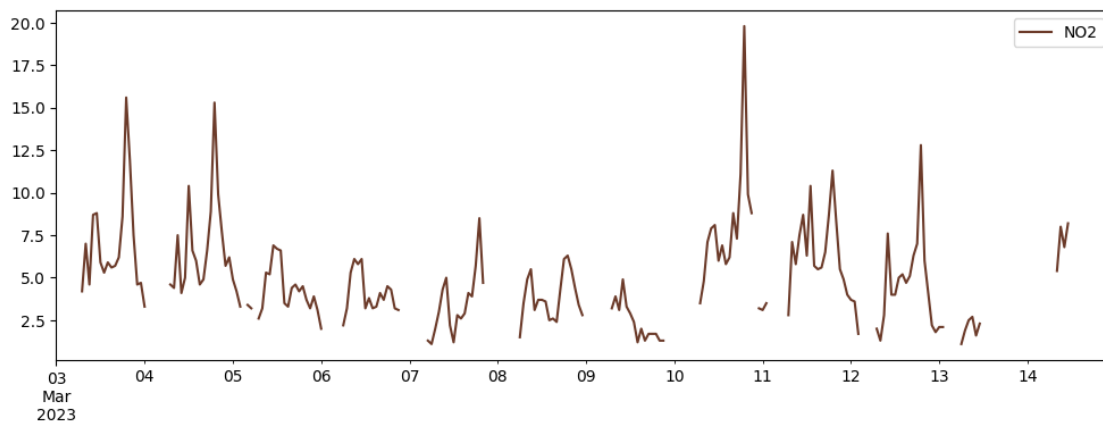
```
[40]: periodo_1_median_imputed.loc["2023-03-03":"2023-03-14"][["NO2"]].
    ↪plot(color='black')
```

[40]: <Axes: >



```
[41]: periodo_1_df.loc["2023-03-03":"2023-03-14"][["NO2"]].plot(color='#6c3b2a')
```

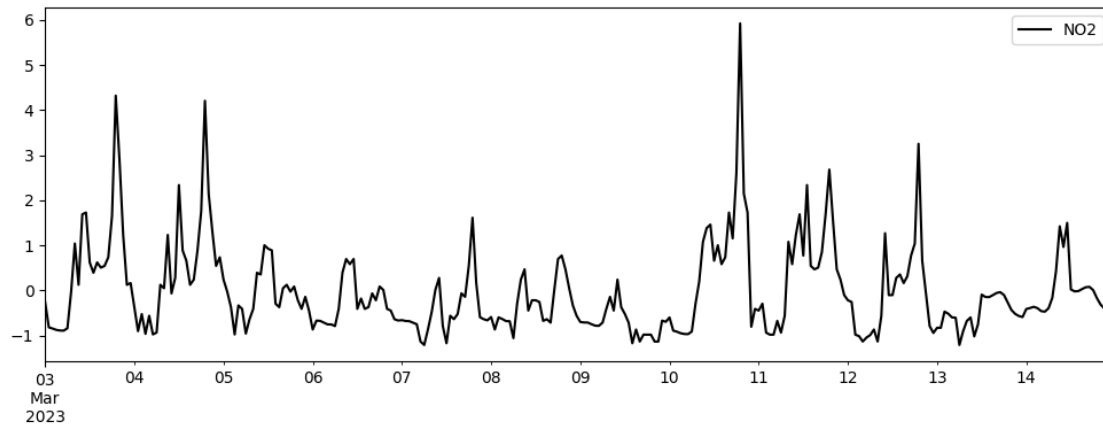
```
[41]: <Axes: >
```



1.16 Serie Temporal NO2 imputada mediante SAITS

```
[42]: periodo_1_imputed.loc["2023-03-03":"2023-03-14"][["NO2"]].plot(color='black')
```

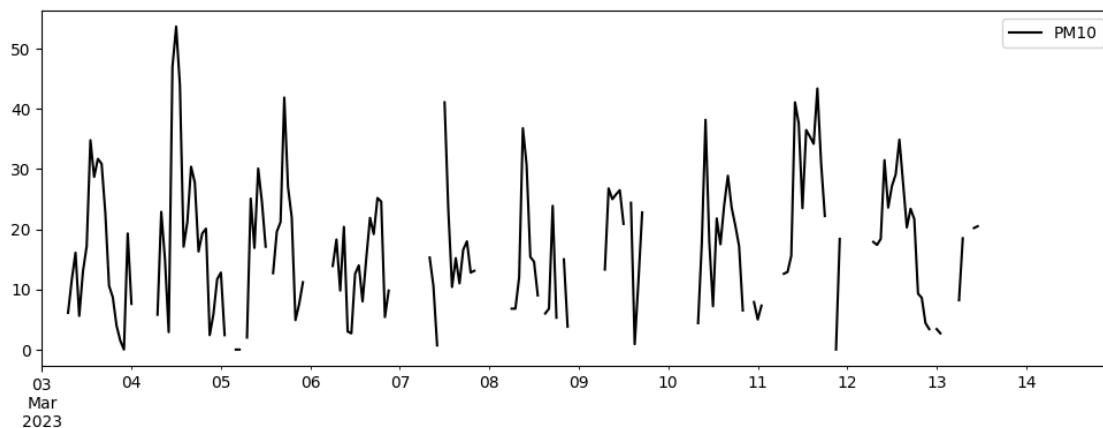
```
[42]: <Axes: >
```

1.17 Serie PM10 con datos faltantes

```
[43]: periodo_1_df.loc["2023-03-03":"2023-03-14"][["PM10"]].plot(color='black')
```

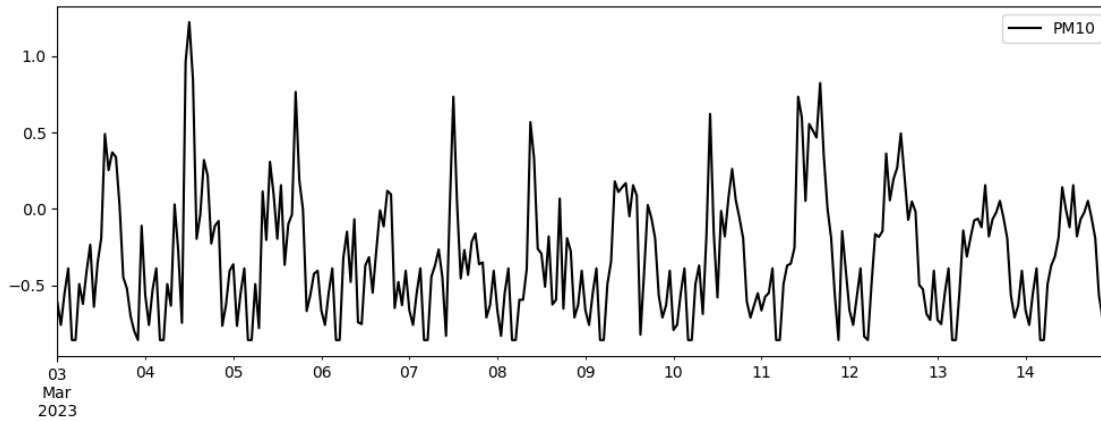
```
[43]: <Axes: >
```



1.18 Serie PM10 imputada mediante la mediana

```
[44]: periodo_1_median_imputed.loc["2023-03-03":"2023-03-14"][["PM10"]].  
      plot(color='black')
```

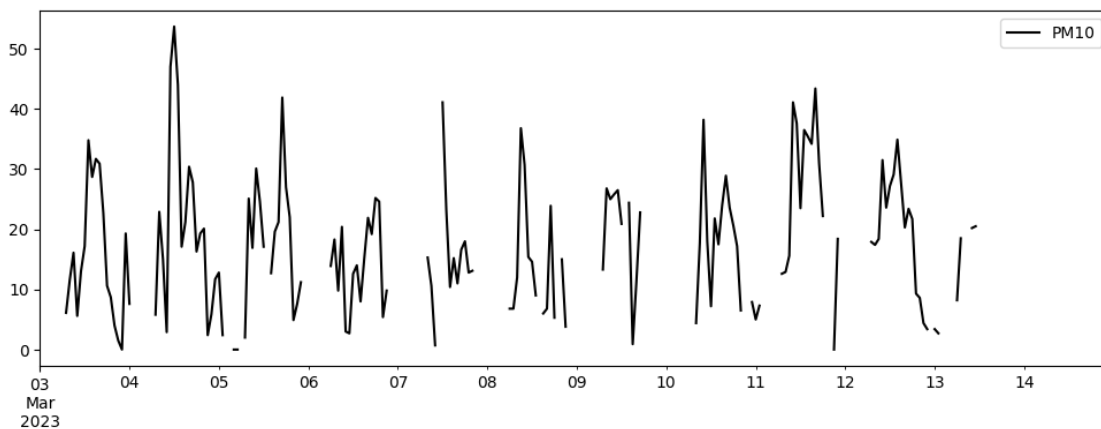
```
[44]: <Axes: >
```



1.19 Serie PM10 con datos faltantes

```
[45]: periodo_1_df.loc["2023-03-03":"2023-03-14"][["PM10"]].plot(color='black')
```

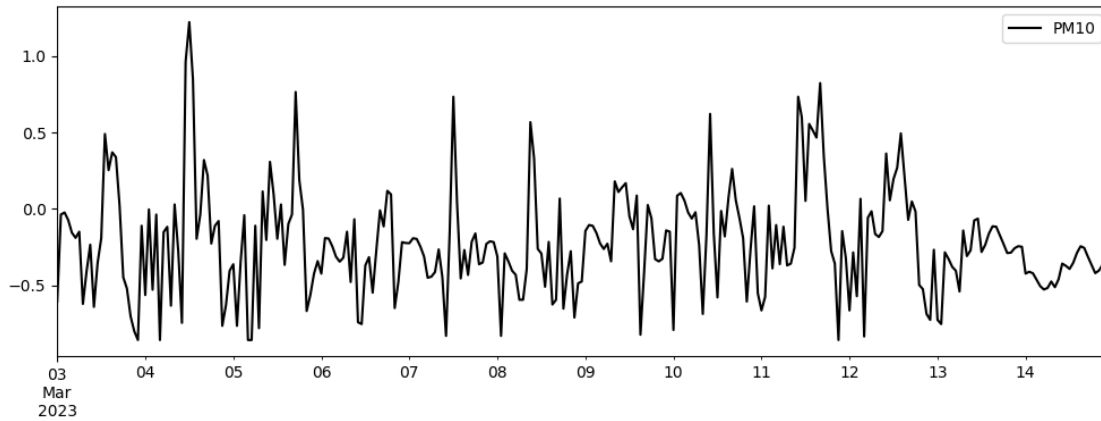
```
[45]: <Axes: >
```



1.20 Serie PM10 imputada mediante SAITS

```
[46]: periodo_1_imputed.loc["2023-03-03":"2023-03-14"][["PM10"]].plot(color='black')
```

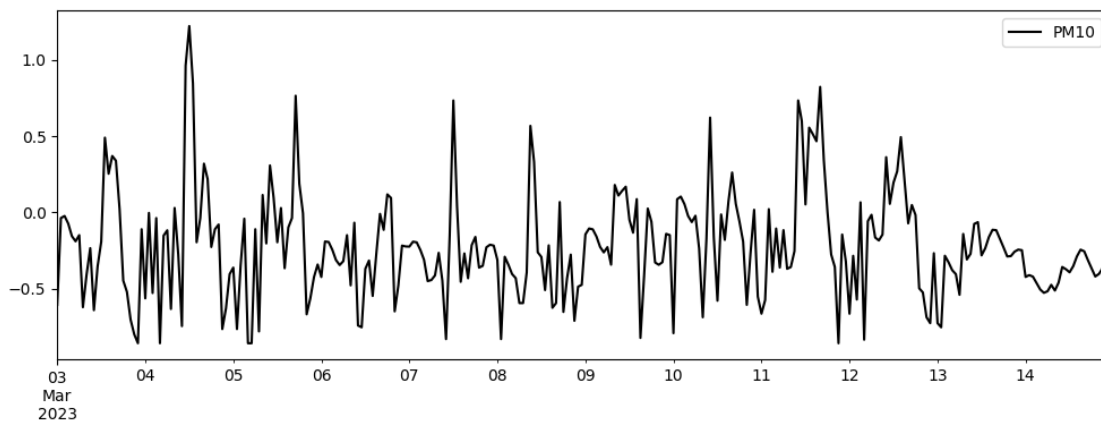
```
[46]: <Axes: >
```



1.21 Serie Temporal NO2 imputada mediante SAITS

```
[47]: periodo_1_imputed.loc["2023-03-03":"2023-03-14"][["PM10"]].plot(color='black')
```

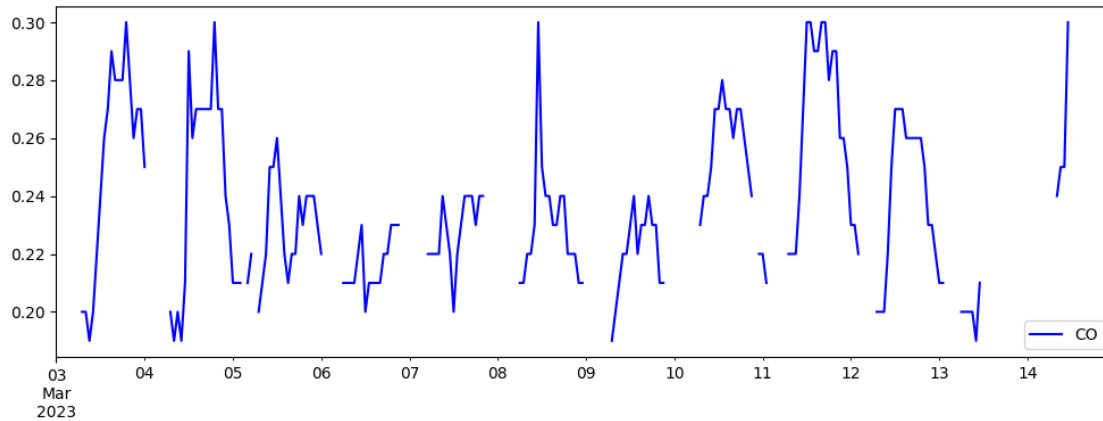
```
[47]: <Axes: >
```



1.22 Serie Temporal CO datos faltantes

```
[48]: periodo_1_df.loc["2023-03-03":"2023-03-14"][["CO"]].plot(color='blue')
```

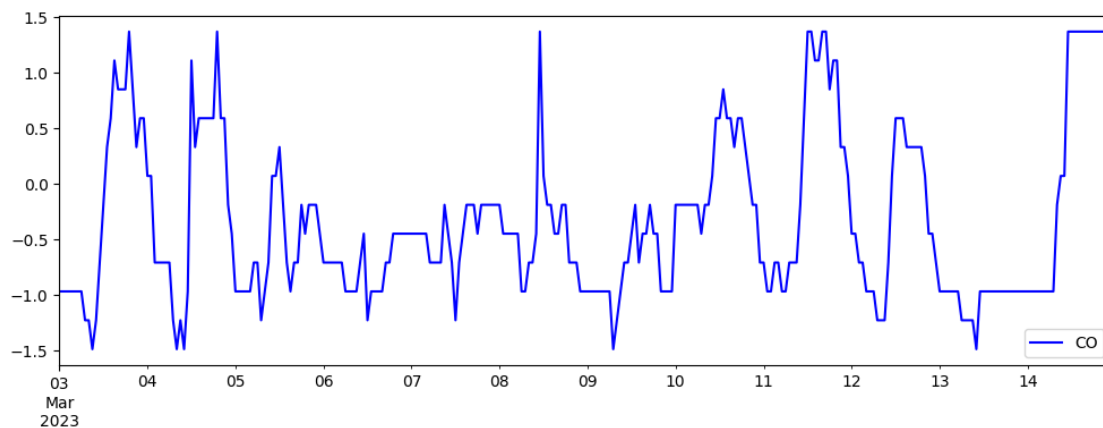
```
[48]: <Axes: >
```



1.23 Serie Temporal CO imputada mediante forward fill

```
[49]: X_imputed_forward[["CO"]].plot(color='blue')
```

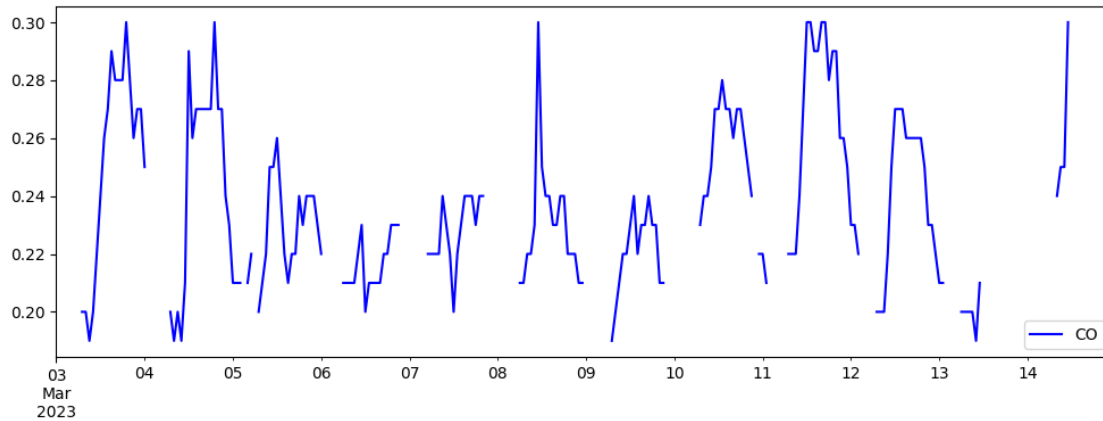
```
[49]: <Axes: >
```



1.24 Serie Temporal CO datos faltantes

```
[50]: periodo_1_df.loc["2023-03-03":"2023-03-14"][["CO"]].plot(color='blue')
```

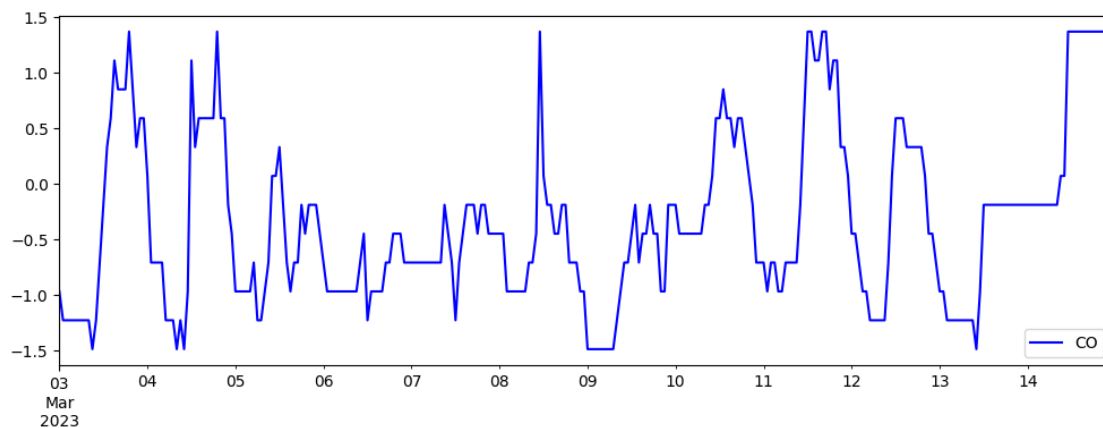
```
[50]: <Axes: >
```



1.25 Serie Temporal CO imputada mediante backwards fill

```
[51]: X_imputed_backwards[["CO"]].plot(color='blue')
```

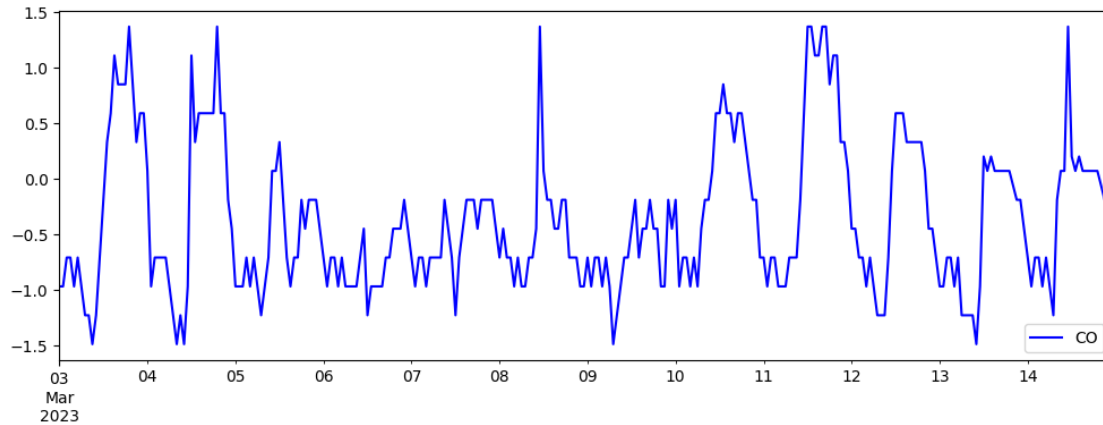
```
[51]: <Axes: >
```



1.26 Serie Temporal CO imputada mediante la mediana

```
[52]: periodo_1_median_imputed.loc["2023-03-03":"2023-03-14"][["CO"]].  
      ↪ plot(color='blue')
```

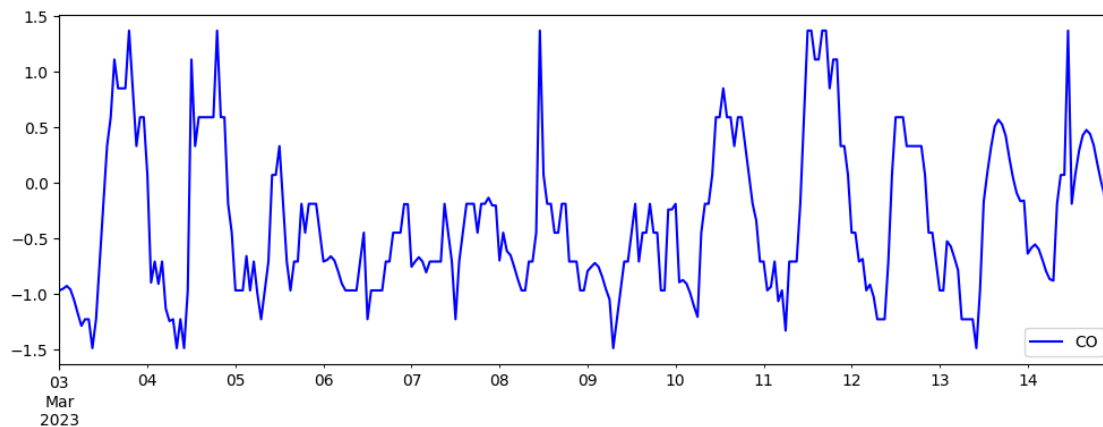
```
[52]: <Axes: >
```



1.27 Serie Temporal CO imputada mediante SAITS

```
[53]: periodo_1_imputed.loc["2023-03-03":"2023-03-14"][["CO"]].plot(color='blue')
```

```
[53]: <Axes: >
```



2 Comparación de SAITS con métodos baseline

2.1 Imputación con la mediana

```
[54]: from pypots.nn.functional import calc_mae, calc_mse, calc_rmse, calc_mre

periodo_1_dataset['X Indicating mask'] = np.isnan(dataset_for_testing['X']).
    astype(int)

# Crear una copia de los datos originales
```

```

X_ori_fixed = np.copy(periodo_1_dataset['test_X_ori'])

# Reemplazar NaNs por 0 (valor constante)
X_ori_fixed[np.isnan(X_ori_fixed)] = 0

# Verificar si quedan NaNs
num_nans_fixed = np.isnan(X_ori_fixed).sum()
print(f"Número de NaNs en test_X_ori después de imputación (con 0):  

↳{num_nans_fixed}")

# Calculate Mean Squared Error (MSE)
mtesting_mae = calc_mae(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Squared Error (MSE)
mtesting_mse = calc_mse(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Root Mean Squared Error (RMSE)
mtesting_rmse = calc_rmse(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Relative Error (MRE)
mtesting_mre = calc_mre(
    X_imputed,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Print the errors
print(f"Testing Mean Absolute Error (MAE): {mtesting_mae:.4f}")
print(f"Testing Mean Squared Error (MSE): {mtesting_mse:.4f}")
print(f"Testing Root Mean Squared Error (RMSE): {mtesting_rmse:.4f}")
print(f"Testing Mean Relative Error (MRE): {mtesting_mre:.4f}")

```

Número de NaNs en test_X_ori después de imputación (con 0): 0
 Testing Mean Absolute Error (MAE): 0.6138
 Testing Mean Squared Error (MSE): 0.4924
 Testing Root Mean Squared Error (RMSE): 0.7017

Testing Mean Relative Error (MRE): 487.5823

2.2 Imputación con la media

```
[55]: from pypots.nn.functional import calc_mae, calc_mse, calc_rmse, calc_mre

# Aplanar 'X_indicating_mask' a la forma (288, 114)
X_indicating_mask_flat = periodo_1_dataset['X_indicating_mask'].reshape(-1,
    len(features))

# Asegurarse de que X_ori_fixed y periodo_1_mean_imputed tengan la misma forma
X_ori_fixed_flat = X_ori_fixed.reshape(-1, len(features)) # Aplanar para que
    tenga la forma (288, 114)

# Convertir periodo_1_mean_imputed a numpy array si es un DataFrame
periodo_1_mean_imputed = periodo_1_mean_imputed.to_numpy() if
    isinstance(periodo_1_mean_imputed, pd.DataFrame) else periodo_1_mean_imputed

# Verificar que ambas matrices tienen la misma forma
assert periodo_1_mean_imputed.shape == X_ori_fixed_flat.shape ==
    X_indicating_mask_flat.shape, (
        f"Las formas no coinciden: {periodo_1_mean_imputed.shape}, "
        f"{X_ori_fixed_flat.shape}, {X_indicating_mask_flat.shape}"
    )

# Calculate Mean Absolute Error (MAE)
mean_testing_mae = calc_mae(
    periodo_1_mean_imputed,
    X_ori_fixed_flat,
    X_indicating_mask_flat,
)

# Calculate Mean Squared Error (MSE)
mean_testing_mse = calc_mse(
    periodo_1_mean_imputed,
    X_ori_fixed_flat,
    X_indicating_mask_flat,
)

# Calculate Root Mean Squared Error (RMSE)
mean_testing_rmse = calc_rmse(
    periodo_1_mean_imputed,
    X_ori_fixed_flat,
    X_indicating_mask_flat,
)

# Calculate Mean Relative Error (MRE)
```



```

mean_testing_mre = calc_mre(
    periodo_1_mean_imputed,
    X_ori_fixed_flat,
    X_indicating_mask_flat,
)

# Print the errors
print(f"Testing Mean Absolute Error (MAE): {mean_testing_mae:.4f}")
print(f"Testing Mean Squared Error (MSE): {mean_testing_mse:.4f}")
print(f"Testing Root Mean Squared Error (RMSE): {mean_testing_rmse:.4f}")
print(f"Testing Mean Relative Error (MRE): {mean_testing_mre:.4f}")

```

```

Testing Mean Absolute Error (MAE): 0.6267
Testing Mean Squared Error (MSE): 0.5179
Testing Root Mean Squared Error (RMSE): 0.7197
Testing Mean Relative Error (MRE): 497.8195

```

2.3 Forward Fill y Backwards Fill

```

[56]: from pypots.nn.functional import calc_mae, calc_mse, calc_rmse, calc_mre
import numpy as np
import pandas as pd

features_fill = [feature for feature in features if feature not in ["WS", "WD"]]
# Crear una copia de los datos originales
X_ori_fixed = np.copy(periodo_1_dataset['test_X_ori'])
X_ori_fixed = np.delete(X_ori_fixed, [111,112],axis=2)

X_indicating_mask_cleaned = np.delete(periodo_1_dataset['X_indicating_mask'],
    ↪[111, 112], axis=2)

# 1. Aplanar 'X_indicating_mask'
X_indicating_mask_flat = X_indicating_mask_cleaned.reshape(-1,
    ↪len(features_fill))
# 1. Aplanar 'X_ori_fixed'

X_ori_fixed_flat = X_ori_fixed.reshape(-1, len(features_fill))
# Reemplazar NaNs por 0 (valor constante)
X_ori_fixed_flat[np.isnan(X_ori_fixed_flat)] = 0

# 3. Convertir 'X_imputed_forward_clean' y 'X_imputed_backwards_clean' a numpy
    ↪arrays si es necesario
if isinstance(X_imputed_forward_clean, pd.DataFrame):
    X_imputed_forward_clean = X_imputed_forward_clean.to_numpy()
if isinstance(X_imputed_backwards_clean, pd.DataFrame):
    X_imputed_backwards_clean = X_imputed_backwards_clean.to_numpy()

```

```

#4 Aplanar forward y backwards
X_imputed_forward_clean = X_imputed_forward_clean.reshape(-1,
↳len(features_fill))
X_imputed_backwards_clean = X_imputed_backwards_clean.reshape(-1,
↳len(features_fill))

def calculate_metrics(y_true, y_pred, mask):
    """Calcula MAE, MSE, RMSE y MRE.

    Args:
        y_true: Valores reales.
        y_pred: Valores predichos.
        mask: Máscara para filtrar valores.

    Returns:
        Una tupla con (rmse, mse, mae, mre).
    """
    mae = calc_mae(y_pred, y_true, mask)
    mse = calc_mse(y_pred, y_true, mask)
    rmse = calc_rmse(y_pred, y_true, mask)
    mre = calc_mre(y_pred, y_true, mask)
    return rmse, mse, mae, mre

# Calcular las métricas para cada método y guardarlas en variables
mean_testing_rmse_forward, mean_testing_mse_forward, mean_testing_mae_forward,
↳mean_testing_mre_forward= calculate_metrics(X_ori_fixed_flat,
↳X_imputed_forward_clean, X_indicating_mask_flat)

mean_testing_rmse_backwards, mean_testing_mse_backwards,
↳mean_testing_mae_backwards, mean_testing_mre_backwards =
↳calculate_metrics(X_ori_fixed_flat, X_imputed_backwards_clean,
↳X_indicating_mask_flat)

# Imprimir las métricas si es necesario (opcional)
print("Forward Fill:", mean_testing_rmse_forward, mean_testing_mse_forward,
↳mean_testing_mae_forward, mean_testing_mre_forward)
print("Backward Fill:", mean_testing_rmse_backwards, mean_testing_mse_backwards,
↳mean_testing_mae_backwards, mean_testing_mre_backwards)

```

Forward Fill: 0.7504637865033926 0.5631958948530097 0.6201314930931956
466.12019806696924
Backward Fill: 0.6955948330243507 0.4838521717301743 0.5826835188169122
437.97252715962384

2.4 Imputación con SAITS

```
[57]: from pypots.nn.functional import calc_mae, calc_mse, calc_rmse, calc_mre

periodo_1_dataset['X_indicating_mask'] = np.isnan(periodo_1_dataset['test_X']).
    ↳astype(int)

# Crear una copia de los datos originales
X_ori_fixed = np.copy(periodo_1_dataset['test_X_ori'])

# Reemplazar NaNs por 0 (valor constante)
X_ori_fixed[np.isnan(X_ori_fixed)] = 0

# Verificar si quedan NaNs
num_nans_fixed = np.isnan(X_ori_fixed).sum()
print(f"Número de NaNs en test_X_ori después de imputación (con 0):_
    ↳{num_nans_fixed}")

# Calculate Mean Squared Error (MSE)
testing_mae = calc_mae(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Squared Error (MSE)
testing_mse = calc_mse(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Root Mean Squared Error (RMSE)
testing_rmse = calc_rmse(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Calculate Mean Relative Error (MRE)
testing_mre = calc_mre(
    saits_imputation,
    X_ori_fixed,
    periodo_1_dataset['X_indicating_mask'],
)

# Print the errors
```

```

print(f"Testing Mean Absolute Error (MAE): {testing_mae:.4f}")
print(f"Testing Mean Squared Error (MSE): {testing_mse:.4f}")
print(f"Testing Root Mean Squared Error (RMSE): {testing_rmse:.4f}")
print(f"Testing Mean Relative Error (MRE): {testing_mre:.4f}")

```

Número de NaNs en test_X_ori después de imputación (con 0): 0
 Testing Mean Absolute Error (MAE): 0.4408
 Testing Mean Squared Error (MSE): 0.2488
 Testing Root Mean Squared Error (RMSE): 0.4988
 Testing Mean Relative Error (MRE): 350.1358

```

[62]: import pandas as pd

# Crear una lista de tuplas (nombre del método, rmse, mse, mae, mre)
error_data = [
    ("Median Imputation", mtesting_rmse, mtesting_mse, mtesting_mae,
    ↪mtesting_mre),
    ("Mean Imputation", mean_testing_rmse, mean_testing_mse, mean_testing_mae,
    ↪mean_testing_mre),
    ("Forward Fill", mean_testing_rmse_forward, mean_testing_mse_forward,
    ↪mean_testing_mae_forward, mean_testing_mre_forward),
    ("Backward Fill", mean_testing_rmse_backwards, mean_testing_mse_backwards,
    ↪mean_testing_mae_backwards, mean_testing_mre_backwards),
    ("SAITS Imputation", testing_rmse, testing_mse, testing_mae, testing_mre),
]

# Crear el DataFrame usando pd.DataFrame.from_records()
error_table = pd.DataFrame.from_records(
    error_data,
    columns=["Method", "RMSE", "MSE", "MAE", "MRE"],
    index="Method" # Establecer la columna "Method" como índice
).round(4) #Redondear los valores a 4 decimales

# Mostrar la tabla transpuesta para mejor visualización
error_table.T

```

```

[62]: Method  Median Imputation  Mean Imputation  Forward Fill  Backward Fill  \
RMSE                0.7017          0.7197          0.7505          0.6956
MSE                 0.4924          0.5179          0.5632          0.4839
MAE                 0.6138          0.6267          0.6201          0.5827
MRE                 487.5823        497.8195        466.1202        437.9725

Method  SAITS Imputation
RMSE                0.4988

```

MSE	0.2488
MAE	0.4408
MRE	350.1358